

Advanced Regression methods Portfolio

Table of contents

Regression splines and smoothing splines	2
Content summary and notes	2
Penalized splines	3
Basis functions for periodic data	3
Smoothing splines	3
Performance measures	3
Tutorial 1	4
Question 1 - Simulating data and fitting various splines	4
Question 2	34
Question 5: proof of a cubic spline	36
Reflection	38
Multidimensional Splines	38
Tutorial 2	38
Fit tensor product splines	42
Fit thin plate splines	45
Fit tensor product splines with custom function	51
Fit thin plate splines with custom function	54
Exercise 1	64
Gams	64
Mini project	64
Summary	64
Summary	64
Reflection	65
MARS	65
Mars tutorial Q1: earth package exploration	65
Mars tutorial Q2: function for basis function construction	71
Mars - analysis of swiss data (numeric Y)	72
Mars - analysis of iris data (categorical Y)	74
how does the mars algorithm accommodate for categorical variables?	74
General	75
Mini project 2	75
Summary	75
Reflection	76
Wavelets	76
Notes	76
Wavelet basis functions:	77
Derivation of daughter	77

General	79
Practice	81
Question 1	81
Difference Between the Two Plots	81
Question 2	83
Question 3	84
Gaussian processes	85
Notes	85
The Covariance Function (Kernel)	85
Comparison with Ordinary Regression	86
A Gaussian Process as a Random Function	87
Kriging	87
Applications of Gaussian Processes	87
Practice	88
Question 1	88
Question 2	91
Regression	95
Notes	95
Mini project 3	97
Summary	97
Reflection	98
Functional Data analysis	99
Notes	99
Mini project 4	101
Summary	101
Reflection	102
References	103
Regression	103
FDA	103

My primary way of grasping new concepts is by application. Therefore, this document will consist primarily of practice questions and exercises. Note-making is also beneficial. When summarizing and relaying information, misunderstandings and other questions are brought to the surface.

Regression splines and smoothing splines

Content summary and notes

- Standard regression models, such as linear and polynomial models, are already familiar.
- Knots ($\xi_1, \dots, \xi_k$) are points along x at which the slope changes and/or at which different polynomial pieces meet.
- Splines are piece-wise polynomial functions joined at knots with continuity up to the $(M - 2)th$ derivative for a spline of order M
- $M = \text{Order} = \text{degree} + 1$ (linear: $M = 2$, quadratic: $M = 3$, cubic: $M = 4$)

- Constructing a basis matrix for different scenarios (piece-wise constant, piece-wise linear, etc) is confusing. I need to become more comfortable with identifying the required basis functions and translating them into the corresponding design matrix.
- Natural cubic splines impose 2 additional constraints: $f''(x) = 0$ for $x < \xi_1$ and $f''(x) = 0$ for $x > \xi_k$. This imposes that the function must be *linear* before the first knot and after the last (ie, the boundary knots). A benefit of this is reduced variance in the context of extrapolating near the boundaries.
- B-splines use Haar basis functions (convolution). It consists of numerous *local* basis functions. My understanding of how the B-spline basis was lacking. I now understand that the reason for the recursive equations is because every prior function is required for the next.
- The problem thus far is that the number and placement of knots are user-selected. Therefore, we explore using large numbers of knots while *regularizing* or *penalizing* the spline to control its flexibility.

Penalized splines

- Penalized regression splines minimizes the β coefficients according to certain constraints. $\|y - X\beta\|^2 + \lambda\beta' D \beta$. Lagrange optimization is applied and results in $\|y - X\beta\|^2 + \lambda\beta' D \beta$ being minimized. D is simply a selection matrix to choose which β coefficients to penalize. For a cubic spline, $1, x, x^2, x^3$ are not penalized. Large values of λ impose a stronger penalty which results in a smoother curve with reduced flexibility (vice versa). This value is chosen via cross validation.
- Alternatively, the penalized error sum of squares (PSS) is minimized: $PSS = \|y - X\beta\|^2 + \lambda\beta' P \beta$ where P is the penalty matrix.
- P-splines combine a B-spline basis and a difference penalty to ensure that neighboring coefficients are similar to each other. This prevents rapid, extreme changes in the fitted curve. The *difference penalty* implemented is: $P = \sum_{i=1}^{k-1} (\beta_{i+1} - \beta_i)^2$.

Basis functions for periodic data

- Fourier series, cyclic splines and radial basis functions can be used for periodic data. Fourier series use sine and cosine basis functions, while cyclic splines impose constraints that ensure the function and its first 2 derivatives are agree at the beginning and end of the cycle.

Smoothing splines

- The optimal curve is a natural cubic spline where every observation is a knot. It is obtained by balancing goodness of fit against a penalty on curvature: $\sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int f''(x)^2 dx$

Performance measures

- Mean Square Error (MSE)
- Mean Square Prediction Error (MSPE)
- Estimates Prediction Error (EPE)
- Leave-One-Out Cross Validation (LOOCV)
- Generalized Cross Validation (GCV) - approximates LOOCV

Tutorial 1

Question 1 - Simulating data and fitting various splines

```
# simulate 100 data points from N(mu_i, 0.25)
x_vec <- numeric(100)
mu_vec <- numeric(100)
y_vec <- numeric(100)

set.seed(8)
x_vec <- runif(100, 0, 1)

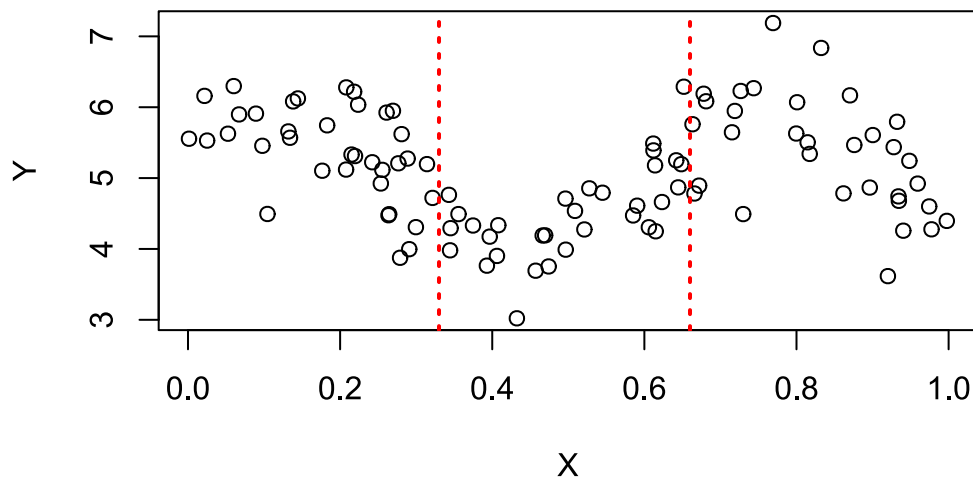
# sort x values for easier plotting later
x_vec <- sort(x_vec)

mu_vec <- 5 + sin(3*pi*(x_vec - 0.6))

knot1 <- 0.33
knot2 <- 0.66

set.seed(8)
y_vec <- rnorm(100, mean = mu_vec, sd = 0.5)

plot(x_vec, y_vec, xlab = "X", ylab = "Y")
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```



```
# knot 1 = 0.33, knot 2 = 0.66
```

```

y_seg1 <- c()
y_seg2 <- c()
y_seg3 <- c()

x_seg1 <- c()
x_seg2 <- c()
x_seg3 <- c()

for (i in 1:100){
  if (x_vec[i] < knot1){
    y_seg1[length(y_seg1)+1] <- y_vec[i]
    x_seg1[length(x_seg1)+1] <- x_vec[i]
  } else{
    if (x_vec[i] < knot2){
      y_seg2[length(y_seg2)+1] <- y_vec[i]
      x_seg2[length(x_seg2)+1] <- x_vec[i]
    } else{
      y_seg3[length(y_seg3)+1] <- y_vec[i]
      x_seg3[length(x_seg3)+1] <- x_vec[i]
    }
  }
}
}

```

```

# piece-wise constant

```

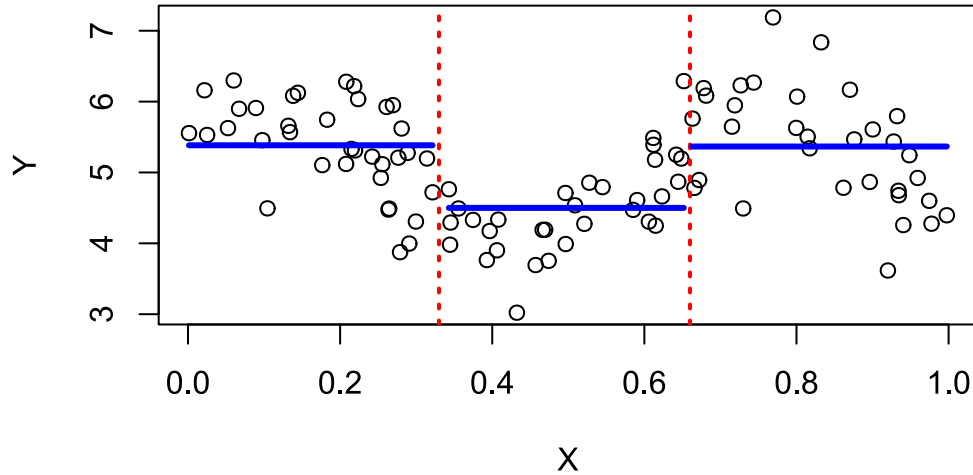
```

y_seg1_pc <- rep(mean(y_seg1), length(y_seg1))
y_seg2_pc <- rep(mean(y_seg2), length(y_seg2))
y_seg3_pc <- rep(mean(y_seg3), length(y_seg3))

plot(x_vec, y_vec, xlab = "X", ylab = "Y", main = "Piecewise constant")
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
lines(x_seg1, y_seg1_pc, lwd = 3, col = "blue")
lines(x_seg2, y_seg2_pc, lwd = 3, col = "blue")
lines(x_seg3, y_seg3_pc, lwd = 3, col = "blue")

```

Piecewise constant



```
# Question -----
#The piece-wise constant function has 3 basis functions. Are each of these the
mean y value of all points within a section, or are the basis functions simply
indicators of whether an observation belongs to the region?

# basis matrix 1: indicator columns only

B1 <- cbind(x_vec < knot1, x_vec >= knot1 & x_vec < knot2, x_vec > knot2) |>
as.matrix()
B1 <- B1 * 1 # change T/F values to numeric 0 and 1

pcl <- lm(y_vec ~ B1 - 1)
pcl_coeff <- pcl$coefficients # coefficients are the means of each segment

# piece wise graphs each overlap their neighbouring segment here, misleading
plot
#plot(x_vec, pcl$fitted.values, pch = '_', cex = 3)
#abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)

# basis matrix 2: column values are the mean values

B2 <- cbind(ifelse(B1[,1], mean(y_seg1), 0), ifelse(B1[,2], mean(y_seg2), 0),
ifelse(B1[,3], mean(y_seg3), 0))
```

```
pc2 <- lm(y_vec ~ B2 - 1)
pc2_coeff <- pc2$coefficients # since values are already the means, the
coefficients are all just 1
```

The -1 is added to the `lm` formula to prevent the algorithm from adding an intercept column. Since we have already added all the required columns in the basis matrix, adding an extra (intercept) column changes the model and misspecifies it.

The basis functions are the indicators of whether an observation is within a certain segment or not. The coefficients will be the mean of those segments. Pre-calculating the means and adding these to the basis matrix forces the coefficients to be 1.

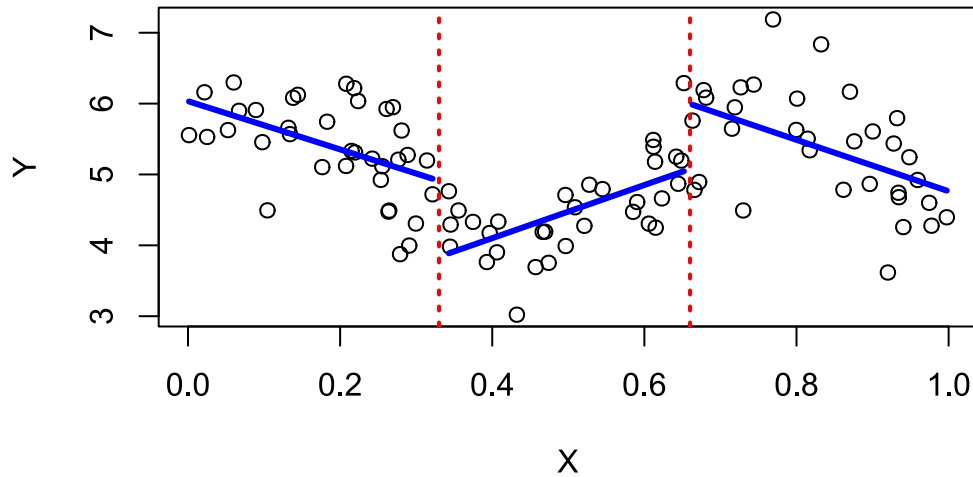
The piece-wise linear function has 6 basis functions: $I(x < k_1)$, $(x - k_1) * I(x < k_1)$, $I(k_1 \leq x < k_2)$, $(x - k_2) * I(k_1 \leq x < k_2)$, $I(x \geq k_2)$, $(x - k_3) * I(x \geq k_2)$. Equivalent basis functions in alternative syntax: $I(x < k_1)$, $(x - k_1)_+$, $I(k_1 \leq x < k_2)$, $(x - k_2)_+$, $I(x \geq k_2)$, $(x - k_3)_+$. This makes use of truncated basis functions. Each segment requires its own intercept and slope.

```
# piece-wise linear (discontinuous)

lm1 <- lm(y_seg1 ~ x_seg1)
lm2 <- lm(y_seg2 ~ x_seg2)
lm3 <- lm(y_seg3 ~ x_seg3)

plot(x_vec, y_vec, xlab = "X", ylab = "Y", main = "Piecewise linear
(discontinuous)")
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
lines(x_seg1, lm1$fitted.values, lwd = 3, col = "blue")
lines(x_seg2, lm2$fitted.values, lwd = 3, col = "blue")
lines(x_seg3, lm3$fitted.values, lwd = 3, col = "blue")
```

Piecewise linear (discontinuous)



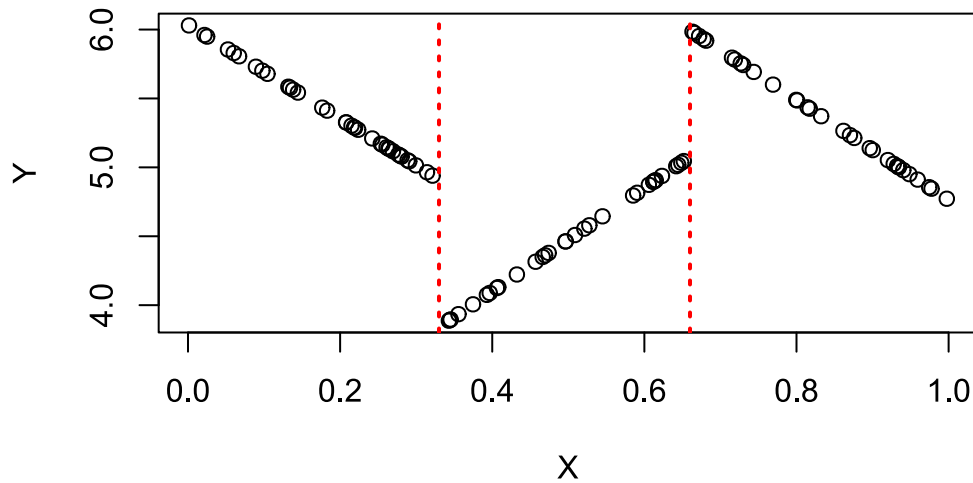
```
indicator_cols <- cbind(ifelse(x_vec < knot1, 1, 0), ifelse(knot1 <= x_vec &
x_vec < knot2, 1, 0), ifelse(x_vec >= knot2, 1, 0))

pld_basis <- cbind (indicator_cols[,1], (x_vec) * indicator_cols[,1],
indicator_cols[,2], (x_vec) * indicator_cols[,2], indicator_cols[,3], (x_vec) *
indicator_cols[,3])

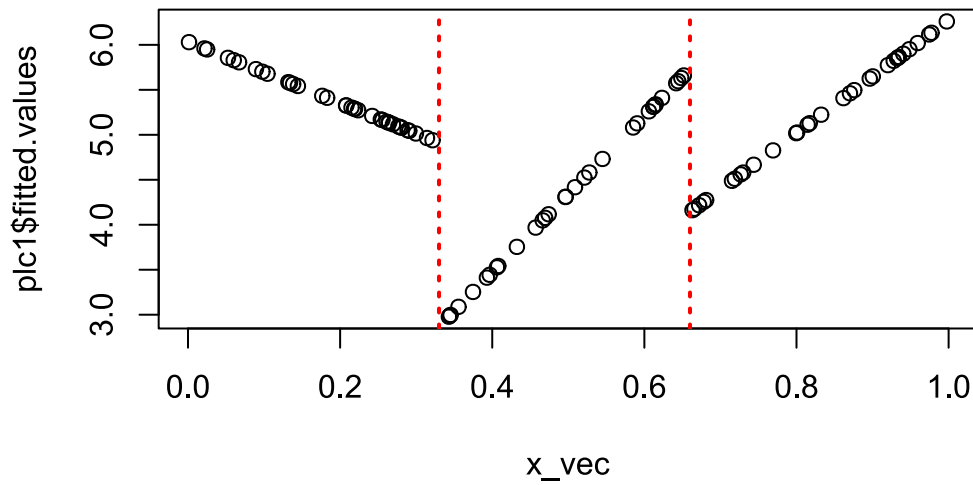
pld <- lm(y_vec ~ pld_basis - 1)
pld_coeff <- pld$coefficients

plot(x_vec, pld$fitted.values, ylab = 'Y', xlab = "X", main = "Piecewise linear
(discontinuous)")
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```

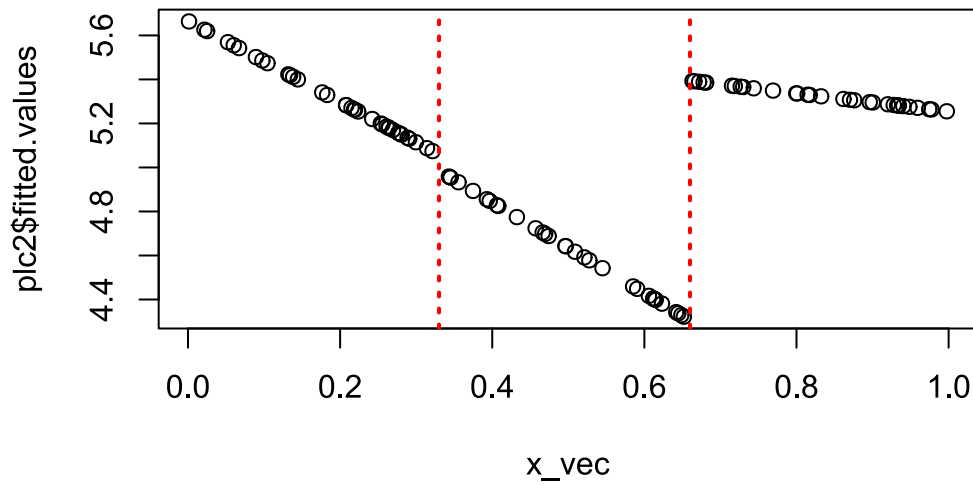
Piecewise linear (discontinuous)



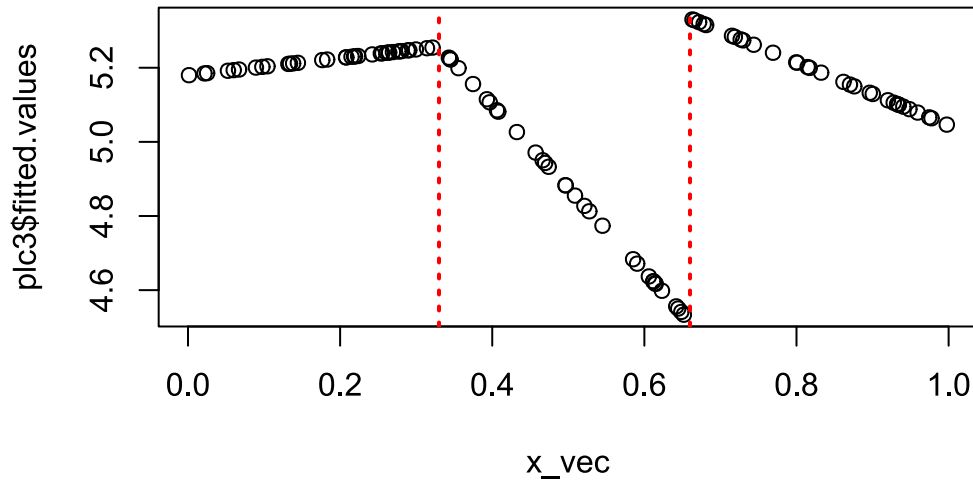
```
# Piecewise linear continuous at knots  
  
# only 1 intercept, 3 truncated basis functions - 1 for each segment  
  
plc_basis1 <- cbind (indicator_cols[,1], (x_vec) * indicator_cols[,1], (x_vec)  
* indicator_cols[,2], (x_vec) * indicator_cols[,3])  
plc1 <- lm(y_vec ~ plc_basis1 - 1)  
plc1_coeff <- plc1$coefficients  
plot(x_vec, plc1$fitted.values)  
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```



```
plc_basis2 <- cbind (1, (x_vec) * indicator_cols[,1], (x_vec) *
indicator_cols[,2], (x_vec) * indicator_cols[,3])
plc2 <- lm(y_vec ~ plc_basis2 - 1)
plc2_coeff <- plc2$coefficients
plot(x_vec, plc2$fitted.values)
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```



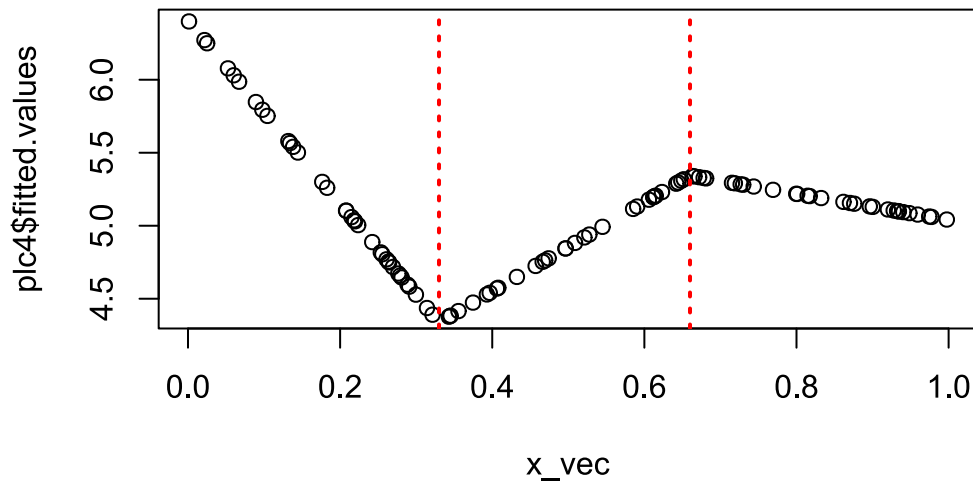
```
plc_basis3 <- cbind(1, (x_vec), (x_vec - knot1) * indicator_cols[,2], (x_vec -
knot2) * indicator_cols[,3])
plc3 <- lm(y_vec ~ plc_basis3 - 1)
plc3_coeff <- plc3$coefficients
plot(x_vec, plc3$fitted.values)
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```



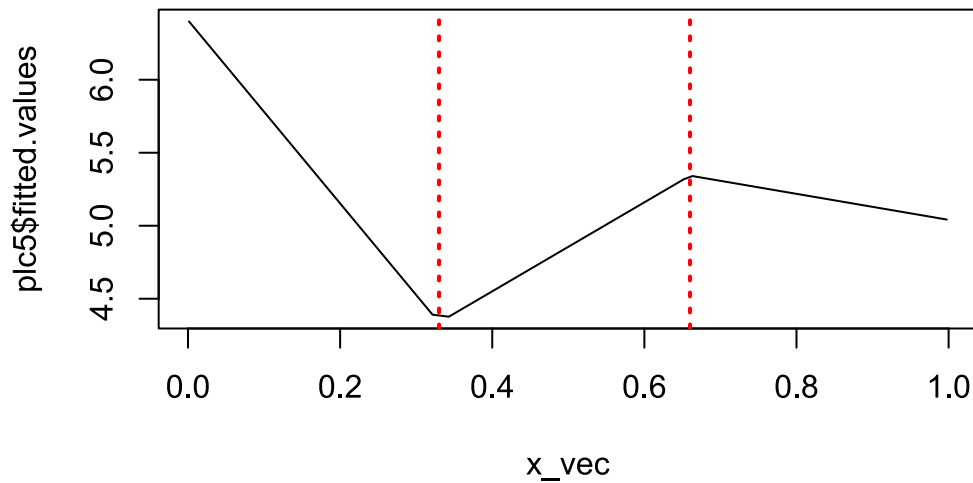
```
# none of the above correctly implement a continuous piecewise linear spline. -
implies error in basis matrix
```

```
segment_cols <- cbind(1, ifelse(knot1 <= x_vec, 1, 0), ifelse(x_vec >= knot2, 1,
0)) # indicator columns for which obs to be used when implementing a continuous
function
```

```
# CORRECT VERSION
plc_basis4 <- cbind(1, (x_vec), (x_vec - knot1) * segment_cols[,2], (x_vec -
knot2) * segment_cols[,3])
plc4 <- lm(y_vec ~ plc_basis4 - 1)
plc4_coeff <- plc4$coefficients
plot(x_vec, plc4$fitted.values)
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```



```
# using truncated basis functions
plc_basis5 <- cbind(1, x_vec, pmax(x_vec - knot1, 0), pmax(x_vec - knot2, 0))
plc5 <- lm(y_vec ~ plc_basis5 - 1)
plc5_coeff <- plc5$coefficients
plot(x_vec, plc5$fitted.values, type = 'l') # plotting line implies that
consecutive observation are joined by a line. This is the reason for the slight
extra line at the first knot.
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```



Plotting type = 'l' (line type, not points) implies that consecutive observations are joined by a line. This is the reason for the slight extra (horizontal) line at the first knot.

For a continuous function, each segment requires all observations that are above the last knot (not just observations that are above last knot and smaller than next knot). This was the issue with the first 3 attempts with the piecewise linear continuous splines (plc1, plc2, plc3)

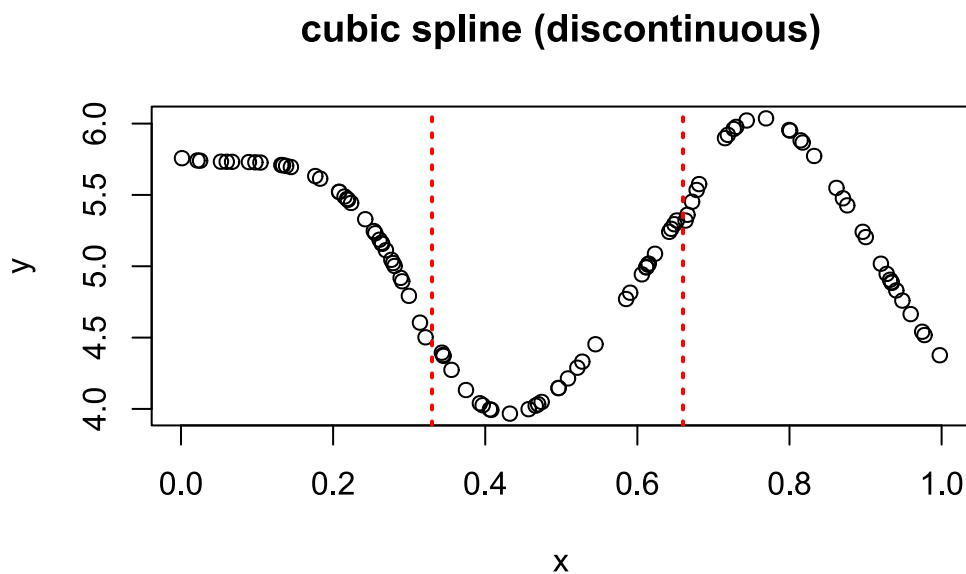
```
# Piecewise cubic splines (discontinuous)

# 12 basis functions - intercept, linear, quadratic and cubic term for each
segment

csd_basis <- cbind(indicator_cols[,1],
  (x_vec) * indicator_cols[,1],
  ((x_vec)^2) * indicator_cols[,1],
  ((x_vec)^3) * indicator_cols[,1],
  indicator_cols[,2],
  (x_vec) * indicator_cols[,2],
  ((x_vec)^2) * indicator_cols[,2],
  ((x_vec)^3) * indicator_cols[,2],
  indicator_cols[,3],
  (x_vec) * indicator_cols[,3],
  ((x_vec)^2) * indicator_cols[,3],
  ((x_vec)^3) * indicator_cols[,3])

csd <- lm(y_vec ~ csd_basis - 1)
```

```
plot(x_vec, csd$fitted.values, main = 'cubic spline (discontinuous)', xlab =
'x', ylab = 'y')
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```



```
# cubic spline (continuous)

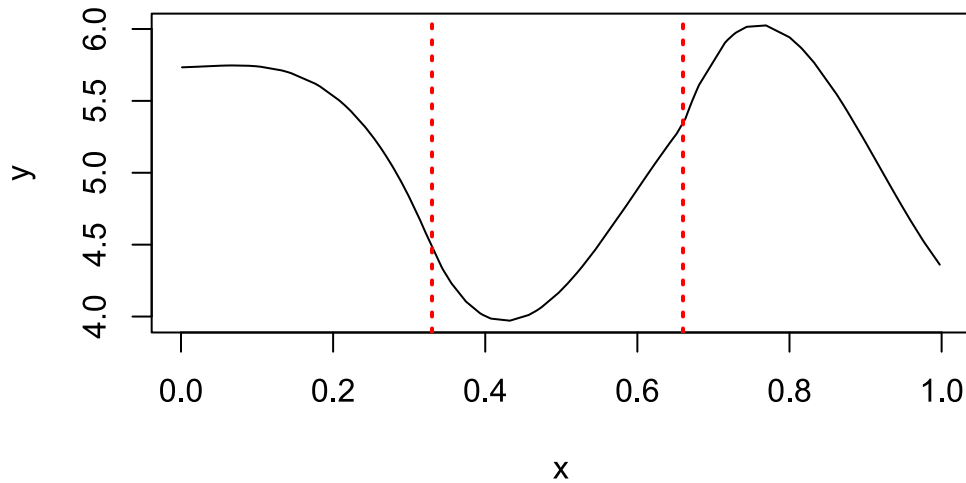
# 10 basis functions: intercept, linear, quad and cubic terms for segment 1,
segments 2 and 3 only require their individual linear, quad and cubic terms.

csc_basis <- cbind(1,
  (x_vec),
  ((x_vec)^2),
  ((x_vec)^3),
  pmax(x_vec - knot1, 0) ,
  (pmax(x_vec - knot1, 0)^2),
  (pmax(x_vec - knot1, 0)^3),
  pmax(x_vec - knot2, 0) ,
  (pmax(x_vec - knot2, 0)^2),
  (pmax(x_vec - knot2, 0)^3))

csc <- lm(y_vec ~ csc_basis - 1)

plot(x_vec, csc$fitted.values, main = 'cubic spline (continuous)', xlab = 'x',
ylab = 'y', type = 'l')
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```

cubic spline (continuous)



```
# cubic spline (continuous first derivative)

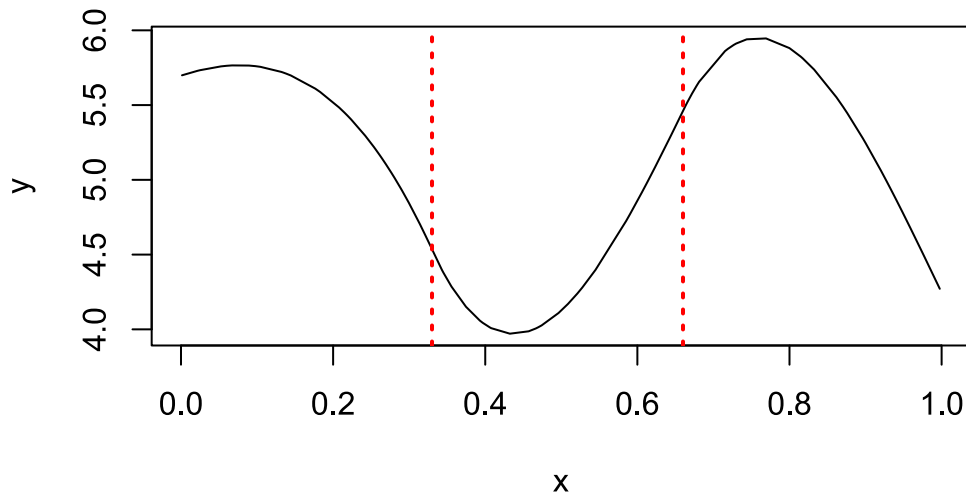
# only 8 basis functions - remove intercepts and linear terms from segments 2
# and 3.

csc_ld_basis <- cbind(1,
                     (x_vec),
                     ((x_vec)^2),
                     ((x_vec)^3),
                     (pmax(x_vec - knot1, 0)^2),
                     (pmax(x_vec - knot1, 0)^3),
                     (pmax(x_vec - knot2, 0)^2),
                     (pmax(x_vec - knot2, 0)^3))

csc_ld <- lm(y_vec ~ csc_ld_basis - 1)

plot(x_vec, csc_ld$fitted.values, main = 'cubic spline (continuous 1st
derivative)', xlab = 'x', ylab = 'y', type = 'l')
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```

cubic spline (continuous 1st derivative)



```
# cubic splines (continuous 2nd derivative)

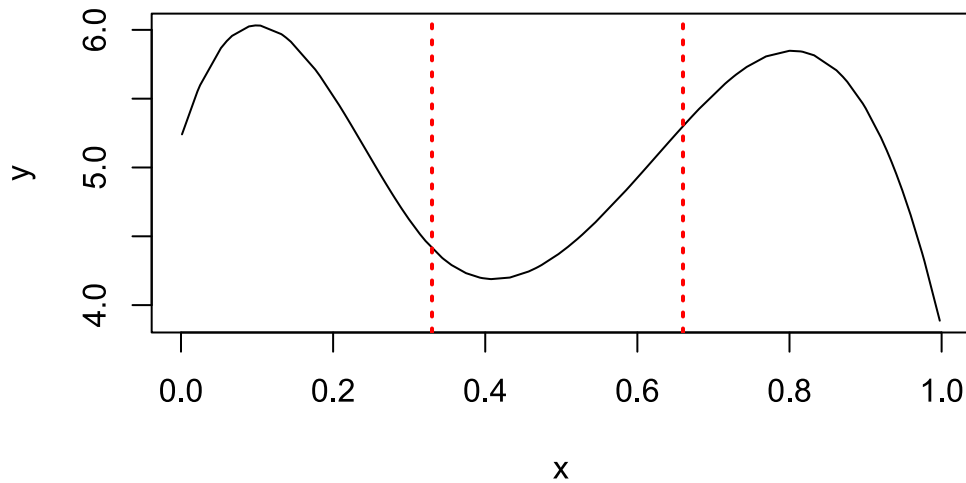
# only 6 basis functions - remove intercepts, linear and quadratic terms from
# segments 2 and 3.
# remaining functions are 1, x, x^2, x^3, (x - k1)^3, (x - k2)^3

csc_2d_basis <- cbind(1,
  (x_vec),
  ((x_vec)^2),
  ((x_vec)^3),
  (pmax(x_vec - knot1, 0)^3),
  (pmax(x_vec - knot2, 0)^3))

csc_2d <- lm(y_vec ~ csc_2d_basis - 1)

plot(x_vec, csc_2d$fitted.values, main = 'cubic spline (continuous 2nd
derivative)', xlab = 'x', ylab = 'y', type = 'l')
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```

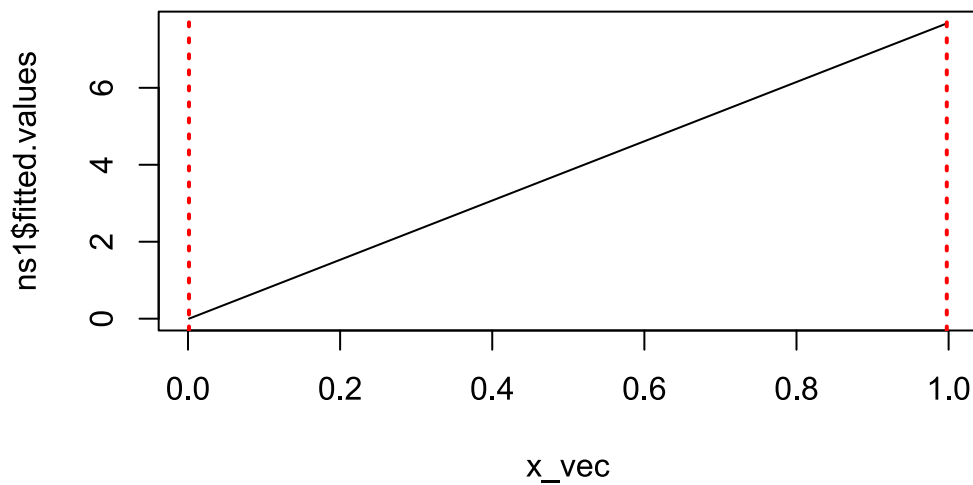
cubic spline (continuous 2nd derivative)



```
# natural splines
library(splines)

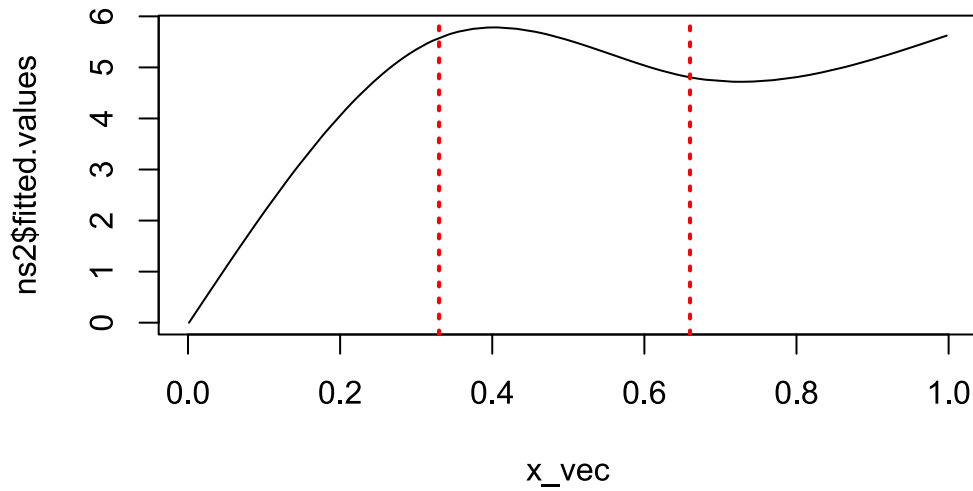
natural.s_basis1 <- ns(x_vec)
att_ns1_b <- attributes(natural.s_basis1)
ns1 <- lm(y_vec ~ natural.s_basis1 - 1)
plot(x_vec, ns1$fitted.values, type = 'l', main = "natural spline 1: Boundary
knots - 0, 1")
abline(v = c(att_ns1_b$Boundary.knots, att_ns1_b$knots), col = 'red', lwd = 2,
lty = 3)
```

natural spline 1: Boundary knots - 0, 1



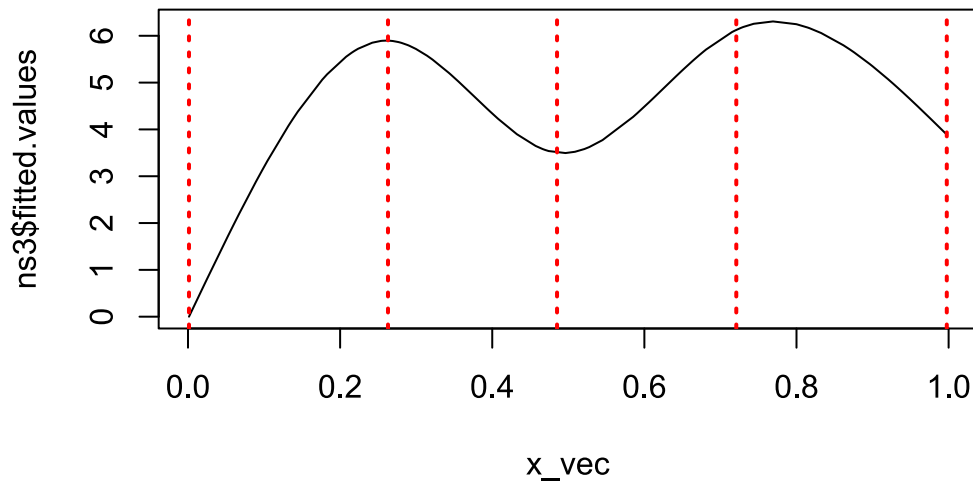
```
#-----  
  
natural.s_basis2 <- ns(x_vec, knots = c(knot1, knot2))  
ns2 <- lm(y_vec ~ natural.s_basis2 - 1)  
plot(x_vec, ns2$fitted.values, type = 'l', main = "natural spline 2: inner knots  
- 0.33, 0.66")  
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```

natural spline 2: inner knots - 0.33, 0.66



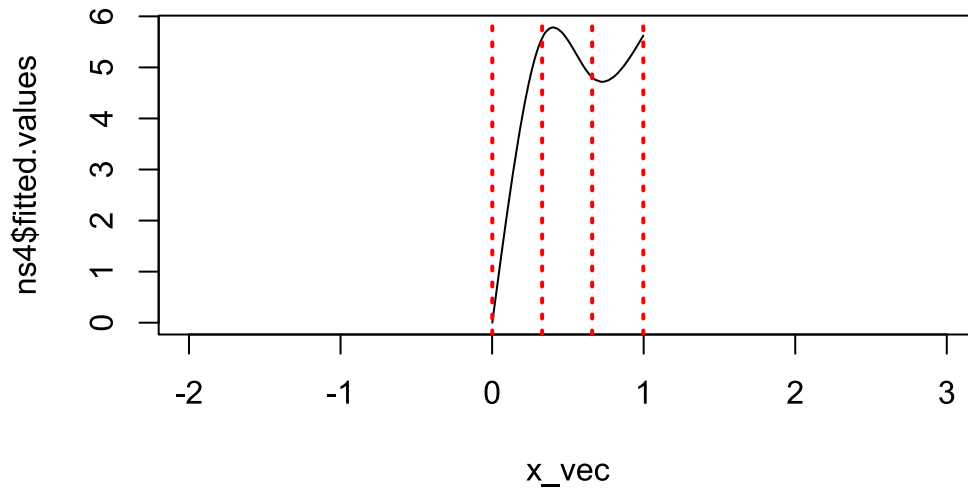
```
#-----  
  
natural.s_basis3 <- ns(x_vec, df = 4)  
att_ns3_b <- attributes(natural.s_basis3)  
ns3 <- lm(y_vec ~ natural.s_basis3 - 1)  
plot(x_vec, ns3$fitted.values, type = 'l', main = "natural spline 3: Dof = 4")  
abline(v = c(att_ns3_b$Boundary.knots, att_ns3_b$knots), col = 'red', lwd = 2,  
lty = 3)
```

natural spline 3: Dof = 4



```
#-----  
  
natural.s_basis4 <- ns(x_vec, knots = c(knot1, knot2))  
att_ns4_b <- attributes(natural.s_basis4)  
ns4 <- lm(y_vec ~ natural.s_basis4 - 1)  
plot(x_vec, ns4$fitted.values, xlim = c(-2, 3), type = 'l', main = "natural  
spline 4: inner knots")  
abline(v = c(att_ns4_b$Boundary.knots, att_ns4_b$knots), col = 'red', lwd = 2,  
lty = 3)
```

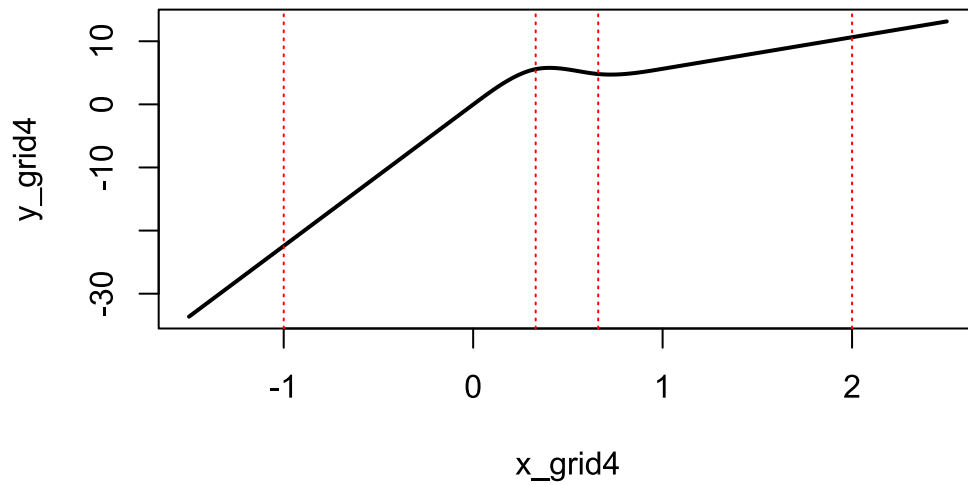
natural spline 4: inner knots



```
# slight experimentation with ns4:
x_grid4 <- seq(-1.5, 2.5, length.out = 500)
B_grid4 <- predict(natural.s_basis4, newx = x_grid4) # predicting using the
basis matrix because matrix used to fit regression model was constructed from
the observed data. Hence, need to first construct the new data matrix. could
also just construct basis matrix of new data, and use this matrix to predict
using the REGRESSION model ns4
y_grid4 <- as.vector(B_grid4 %*% coef(ns4)) # this is manually doing the
regression prediction using the coeff

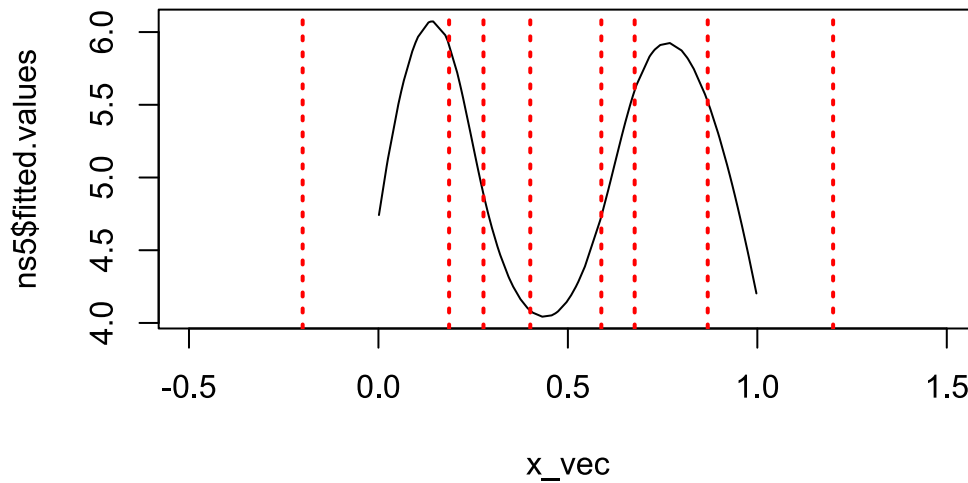
plot(x_grid4, y_grid4, type = "l", xlim = c(-1.5, 2.5), ylim = range(y_grid4),
lwd = 2, main = "natural spline 4: prediction past bounds")
abline(v = c(-1, knot1, knot2, 2), col = "red", lty = 3)
```

natural spline 4: prediction past bounds



```
#-----  
natural.s_basis5 <- ns(x_vec, df = 7, Boundary.knots = c(-0.2, 1.2))  
att_ns5_b <- attributes(natural.s_basis5)  
ns5 <- lm(y_vec ~ natural.s_basis5 - 1)  
plot(x_vec, ns5$fitted.values, xlim = c(-0.5, 1.5), type = 'l', main = "natural  
spline 5: Boundary knots and dof specified")  
abline(v = c(att_ns5_b$Boundary.knots, att_ns5_b$knots), col = 'red', lwd = 2,  
lty = 3)
```

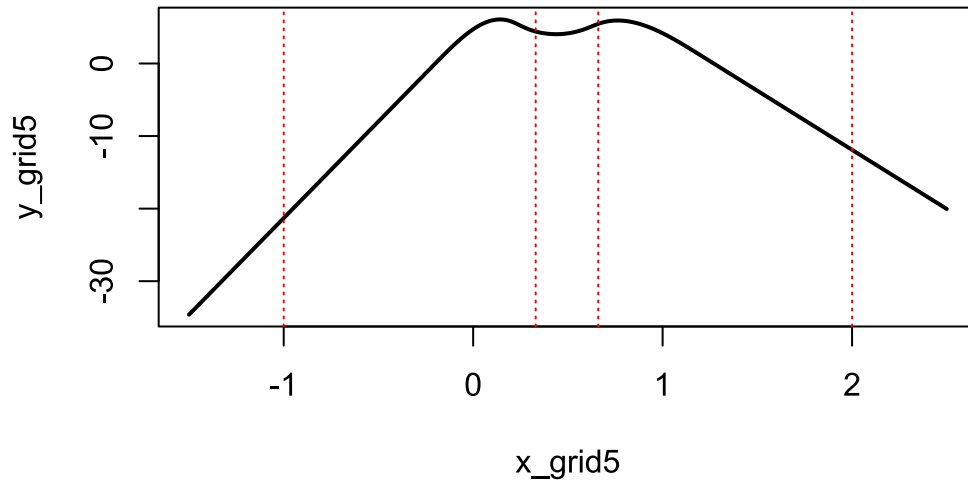
natural spline 5: Boundary knots and dof specified



```
# slight experimentation with ns4:
x_grid5 <- seq(-1.5, 2.5, length.out = 500)
B_grid5 <- predict( natural.s_basis5, newx = x_grid5)
y_grid5 <- as.vector(B_grid5 %*% coef(ns5))

plot(x_grid5, y_grid5, type = "l", xlim = c(-1.5, 2.5), ylim = range(y_grid5),
     lwd = 2, main = "natural spline 5 prediction past bounds")
abline(v = c(-1, knot1, knot2, 2), col = "red", lty = 3)
```

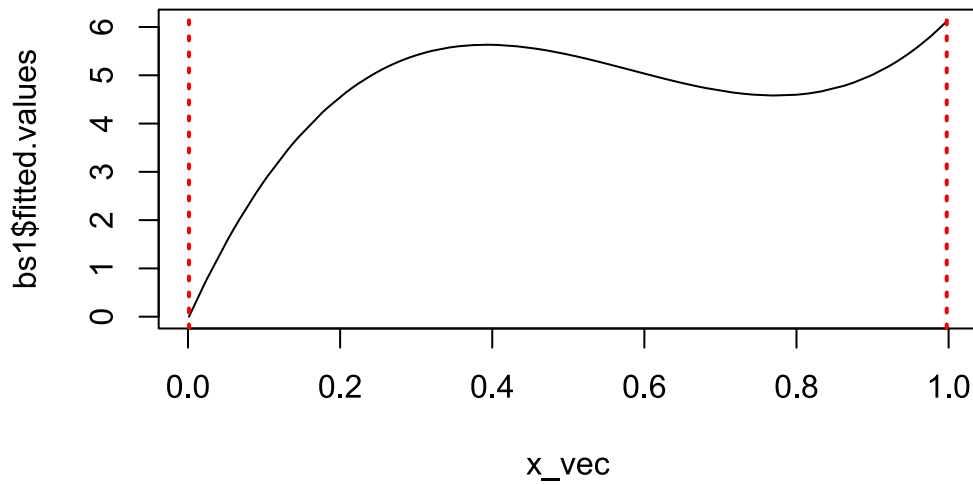
natural spline 5 prediction past bounds



```
# B-splines

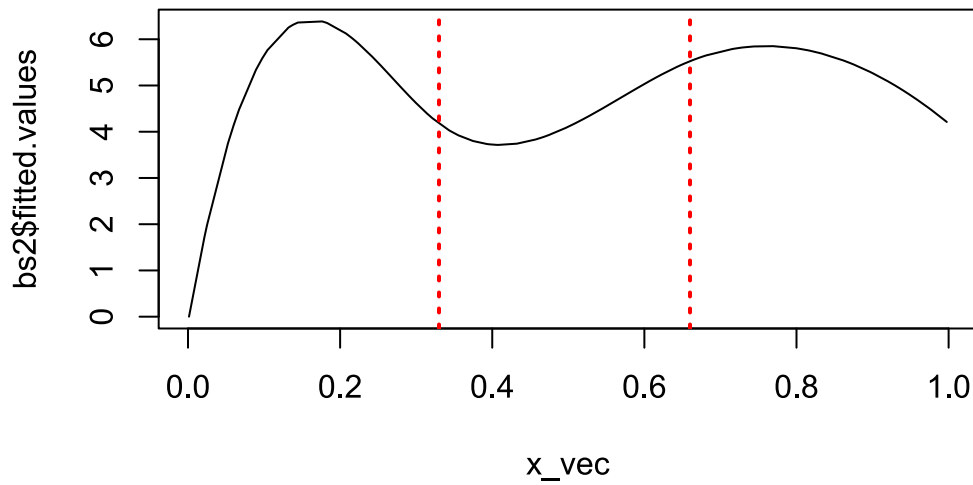
b.s_basis1 <- bs(x_vec)
att_bs1_b <- attributes(b.s_basis1)
bs1 <- lm(y_vec ~ b.s_basis1 - 1)
plot(x_vec, bs1$fitted.values, type = 'l', main = "B spline 1: Boundary knots -
0, 1")
abline(v = c(att_bs1_b$Boundary.knots, att_bs1_b$knots), col = 'red', lwd = 2,
lty = 3)
```

B spline 1: Boundary knots - 0, 1



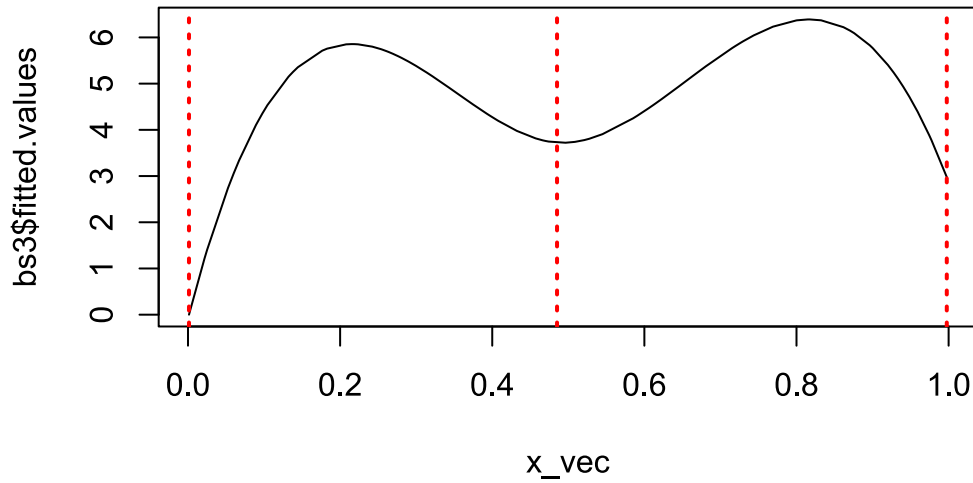
```
#-----  
  
b.s_basis2 <- bs(x_vec, knots = c(knot1, knot2))  
bs2 <- lm(y_vec ~ b.s_basis2 - 1)  
plot(x_vec, bs2$fitted.values, type = 'l', main = "B spline 2: inner knots -  
0.33, 0.66")  
abline(v = c(knot1, knot2), col = 'red', lwd = 2, lty = 3)
```

B spline 2: inner knots - 0.33, 0.66



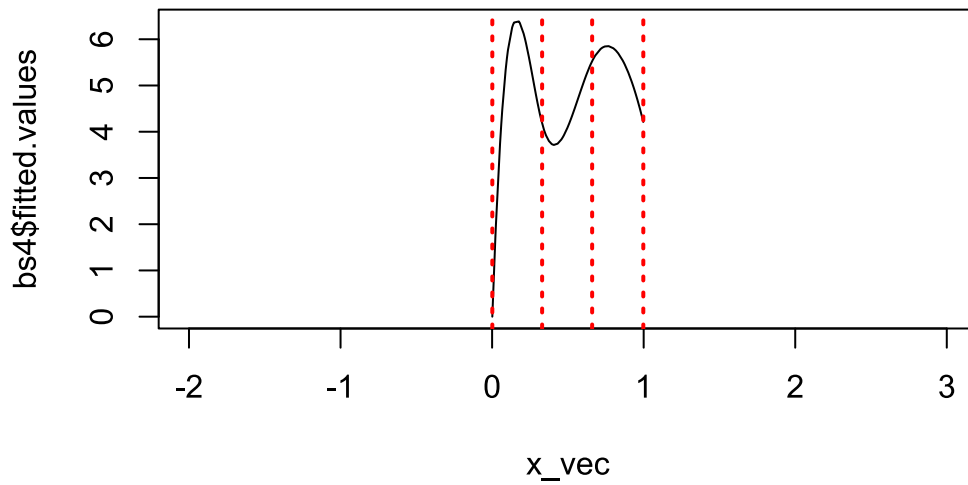
```
#-----  
  
b.s_basis3 <- bs(x_vec, df = 4)  
att_bs3_b <- attributes(b.s_basis3)  
bs3 <- lm(y_vec ~ b.s_basis3 - 1)  
plot(x_vec, bs3$fitted.values, type = 'l', main = "B spline 3: Dof = 4")  
abline(v = c(att_bs3_b$Boundary.knots, att_bs3_b$knots), col = 'red', lwd = 2,  
lty = 3)
```

B spline 3: Dof = 4



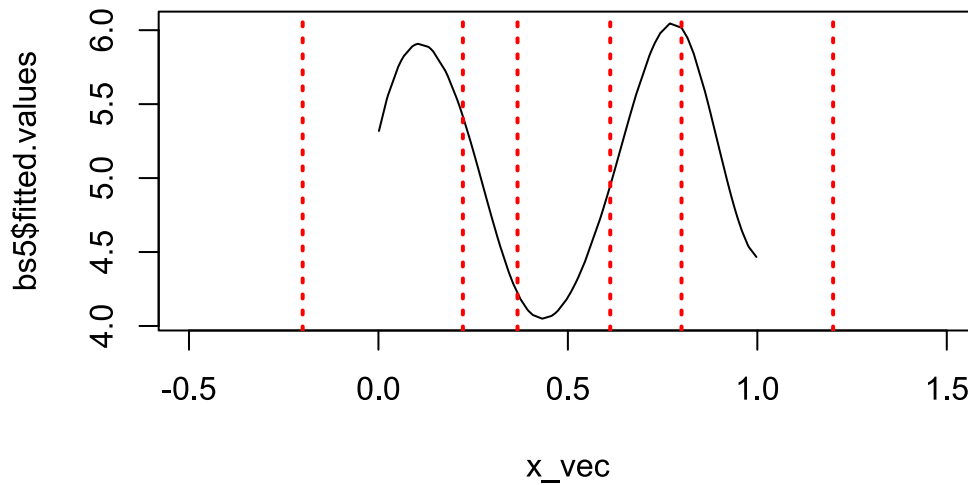
```
#-----  
  
b.s_basis4 <- bs(x_vec, knots = c(knot1, knot2))  
att_bs4_b <- attributes(b.s_basis4)  
bs4 <- lm(y_vec ~ b.s_basis4 - 1)  
plot(x_vec, bs4$fitted.values, xlim = c(-2, 3), type = 'l', main = "B spline 4:  
inner knots")  
abline(v = c(att_bs4_b$Boundary.knots, att_bs4_b$knots), col = 'red', lwd = 2,  
lty = 3)
```

B spline 4: inner knots



```
#-----  
  
b.s_basis5 <- bs(x_vec, df = 7, Boundary.knots = c(-0.2, 1.2))  
att_bs5_b <- attributes(b.s_basis5)  
bs5 <- lm(y_vec ~ b.s_basis5 - 1)  
plot(x_vec, bs5$fitted.values, xlim = c(-0.5, 1.5), type = 'l', main = "B spline  
5: Boundary knots and dof specified")  
abline(v = c(att_bs5_b$Boundary.knots, att_bs5_b$knots), col = 'red', lwd = 2,  
lty = 3)
```

B spline 5: Boundary knots and dof specified

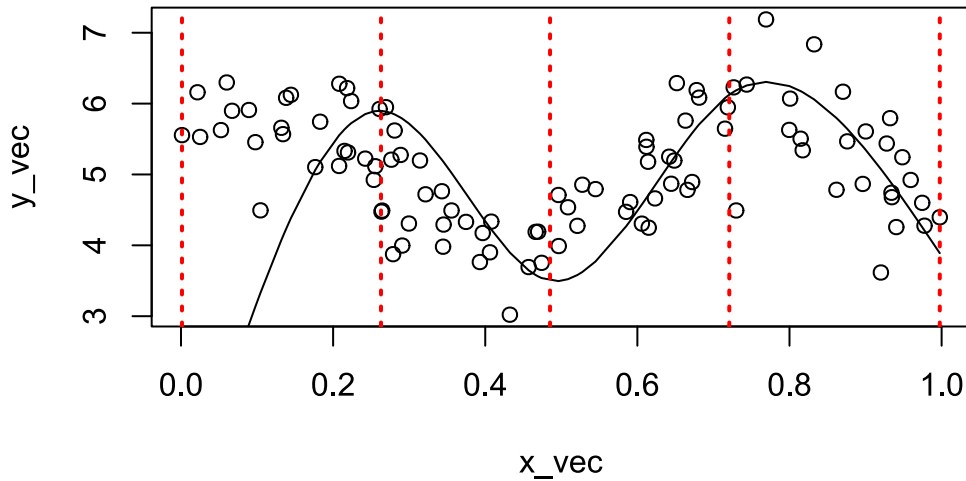


```
# adding confidence intervals to the natural splines

natural.s_basis6 <- ns(x_vec, df = 4)

att_ns6_b <- attributes(natural.s_basis6)
ns6 <- lm(y_vec ~ natural.s_basis6 - 1)
plot(x_vec, y_vec, main = "natural spline 6: 4 Dof specified")
lines(x_vec, ns6$fitted.values)
abline(v = c(att_ns6_b$Boundary.knots, att_ns6_b$knots), col = 'red', lwd = 2,
lty = 3)
```

natural spline 6: 4 Dof specified



```
# Dense grid of x values
x_grid_conf <- seq(min(x_vec), max(x_vec), length.out = 500)

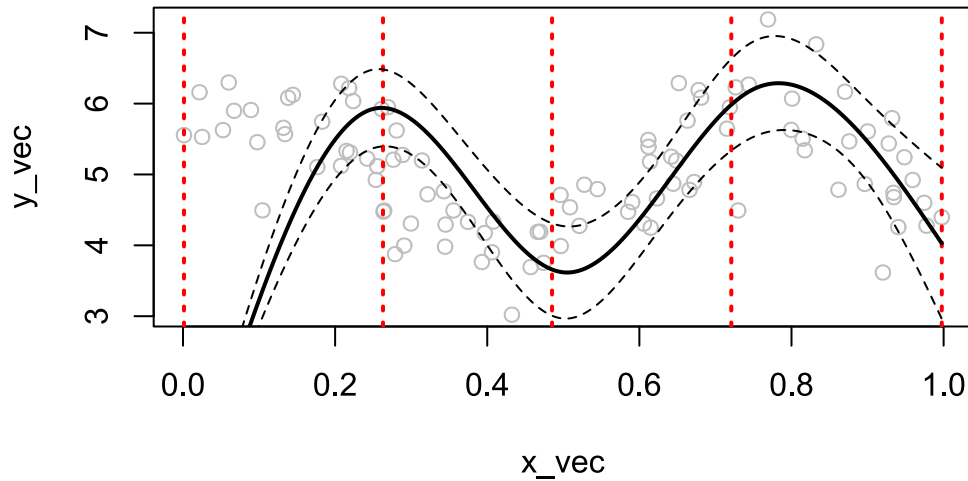
natural.s_basis6_new <- ns(x_grid_conf, df = 4)

#ns4_pred <- predict(ns4, newdata = data.frame(natural.s_basis4 =
natural.s_basis4_new), interval = "confidence")
#ns4_pred <- predict(ns4, newdata = natural.s_basis4_new, interval =
"confidence")

ns6_pred <- predict(ns6, newdata = data.frame(natural.s_basis6 =
I(natural.s_basis6_new)), interval = "confidence")

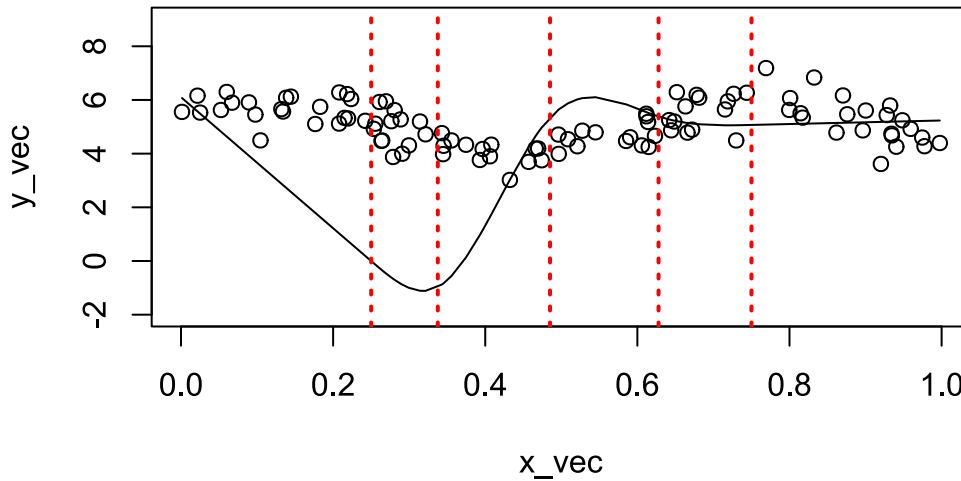
plot(x_vec, y_vec, col = "grey", main = 'natural spline 6: cubic with confidence
interval')
abline(v = c(att_ns6_b$Boundary.knots, att_ns6_b$knots), col = 'red', lwd = 2,
lty = 3)
lines(x_grid_conf, ns6_pred[, "fit"], lwd = 2)
lines(x_grid_conf, ns6_pred[, "lwr"], lty = 2)
lines(x_grid_conf, ns6_pred[, "upr"], lty = 2)
```

natural spline 6: cubic with confidence interval



```
# here, cannot see effect of the natural spline, cannot see linear segments.  
#repeat with another model where boundary knots are within x range  
  
natural.s_basis7 <- ns(x_vec, df = 4, Boundary.knots = c(0.25, 0.75))  
  
att_ns7_b <- attributes(natural.s_basis7)  
ns7 <- lm(y_vec ~ natural.s_basis7 - 1)  
plot(x_vec, y_vec, main = "natural spline 7: 4 Dof and boundary knots  
specified", , ylim = c(-2, 9))  
lines(x_vec, ns7$fitted.values)  
abline(v = c(att_ns7_b$Boundary.knots, att_ns7_b$knots), col = 'red', lwd = 2,  
lty = 3)
```

natural spline 7: 4 Dof and boundary knots specified

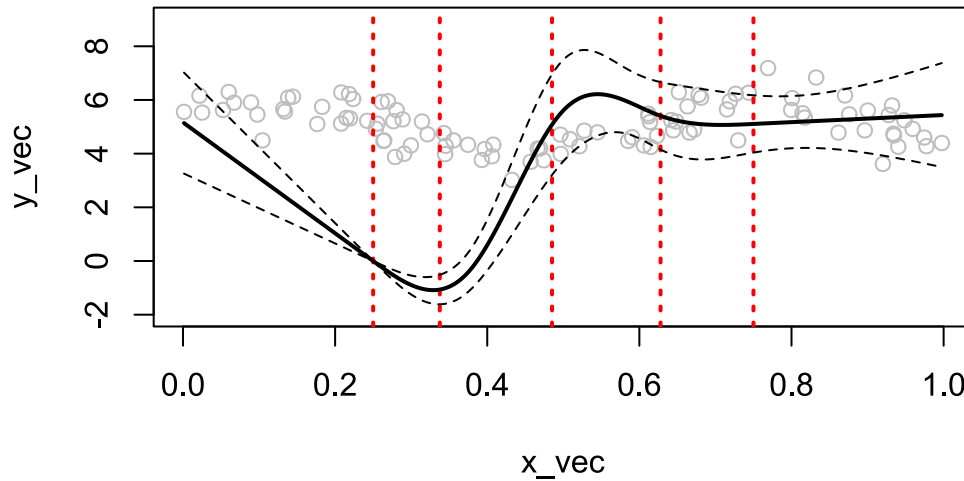


```
natural.s_basis7_new <- ns(x_grid_conf, df = 4, Boundary.knots = c(0.25, 0.75))

ns7_pred <- predict(ns7, newdata = data.frame(natural.s_basis7 =
I(natural.s_basis7_new)), interval = "confidence")

plot(x_vec, y_vec, col = "grey", main = 'natural spline 7: cubic with confidence
interval', ylim = c(-2, 9))
abline(v = c(att_ns7_b$Boundary.knots, att_ns7_b$knots), col = 'red', lwd = 2,
lty = 3)
lines(x_grid_conf, ns7_pred[, "fit"], lwd = 2)
lines(x_grid_conf, ns7_pred[, "lwr"], lty = 2)
lines(x_grid_conf, ns7_pred[, "upr"], lty = 2)
```

natural spline 7: cubic with confidence interval



```
# linear quality of natural splines is evident in the ns7 plots
```

Question 2

```
#library(SemiPar) # not able to be installed
#install.packages("JOPS")
library(JOPS)

data("lidar")

logratio <- lidar$logratio
lidar_x <- lidar$range

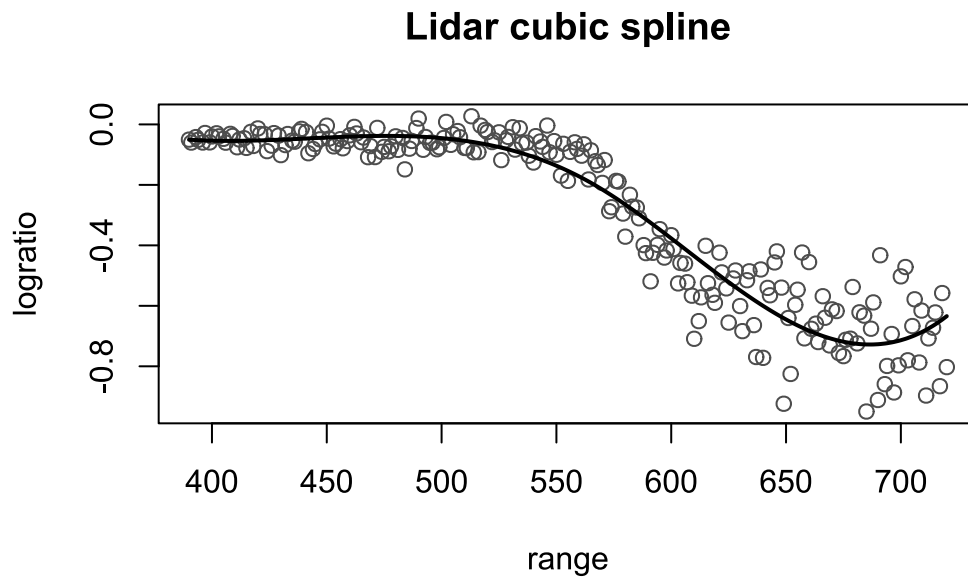
plot(lidar_x, logratio, col = 'grey30', main = "Lidar cubic spline", xlab =
"range", ylab = "logratio")

create_cubic_basis <- function(x_var, knot) {
  basis_mat <- cbind(intercept = 1, x = x_var, x2 = x_var^2, x3 = x_var^3,
trunc_base = pmax((x_var-knot)^3, 0)) |> as.matrix()
  return(basis_mat)
}

lidar_basis <- create_cubic_basis(lidar_x, 565)

lidar_cubic_spline <- lm(logratio ~ lidar_basis - 1)
```

```
lines(lidar$range, lidar_cubic_spline$fitted.values, lwd = 2)
```



```
## to create confidence intervals - warning

#x_lidar_pred <- seq(min(lidar$range), max(lidar$range), length = 500) |>
as.matrix(ncol = 1)
#x_lidar_pred_basis <- create_cubic_basis(x_lidar_pred, 565) |> as.data.frame()

#lidar_pred <- predict(lidar_cubic_spline, newdata = x_lidar_pred_basis,
interval = 'confidence')

#smooth spline

lidar_smooth_spline1 <- smooth.spline(lidar_x, logratio, cv = T)
lidar_smooth_spline2 <- smooth.spline(lidar_x, logratio, lambda = 0)
lidar_smooth_spline3 <- smooth.spline(lidar_x, logratio, lambda = 0.15)
lidar_smooth_spline4 <- smooth.spline(lidar_x, logratio, lambda = 1)

plot(lidar_x, logratio, col = 'grey30', main = "Lidar smooth spline", xlab =
"range", ylab = "logratio")
lines(lidar_smooth_spline1, lwd = 3)
```

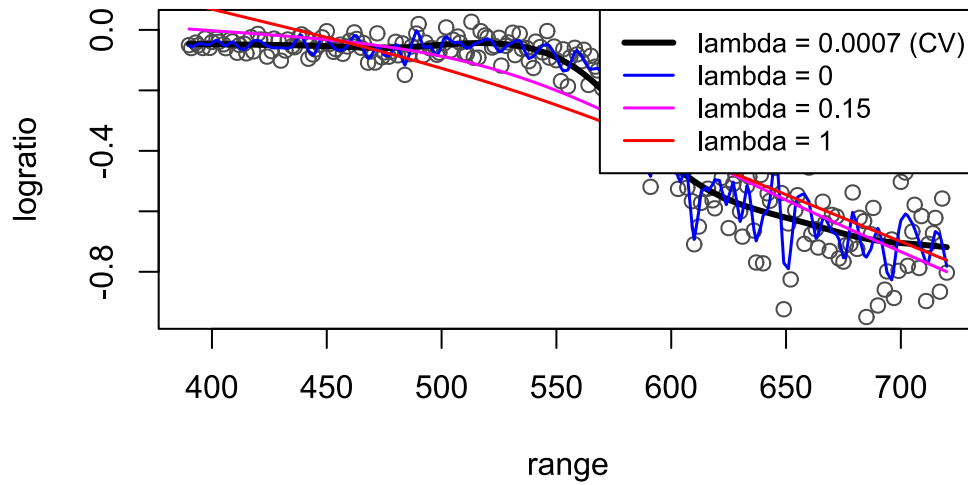
```

lines(lidar_smooth_spline2, lwd = 1.5, col = "blue")
lines(lidar_smooth_spline3, lwd = 1.5, col = "magenta")
lines(lidar_smooth_spline4, lwd = 1.5, col = "red")

legend('topright', col = c('black', 'blue', 'magenta', 'red'), lwd = c(3, 1.5,
1.5, 1.5), legend = c("lambda = 0.0007 (CV)", "lambda = 0", "lambda = 0.15",
"lambda = 1"), cex = 0.85)

```

Lidar smooth spline



Question 5: proof of a cubic spline

Let $f(x)$ be of the form $\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)_+^3$. Let ξ be the one and only knot.

Let $f_1(x)$ be the form of $f(x)$ for when $x \leq \xi$. Since x is not greater than ξ , the last (β_4) term falls away. Therefore, $f_1(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$.

Let $f_2(x)$ be the form of $f(x)$ for when $x > \xi$. Therefore, $f_2(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)^3$.

Expanding the cubic binomial results in

$$\begin{aligned}
 f_2(x) &= \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x^3 - 3x^2\xi + 3x\xi^2 - \xi^3) \\
 &= \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^3 - 3\beta_4 x^2\xi + 3\beta_4 x\xi^2 - \beta_4 \xi^3
 \end{aligned}$$

To write $f_2(x)$ in the form of $a + bx + cx^2 + dx^3$, we group by each power of x (x^0, x^1, x^2, x^3).

$$f_2(x) = (\beta_0 - \beta_4 \xi^3) + (\beta_1 + 3\beta_4 \xi^2)x + (\beta_2 - 3\beta_4 \xi)x^2 + (\beta_3 + \beta_4)x^3$$

We now evaluate each of the functions at $x = \xi$.

$$f_1(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

$$f_1(x = \xi) = \beta_0 + \beta_1 \xi + \beta_2 \xi^2 + \beta_3 \xi^3$$

$$f_2(x) = (\beta_0 - \beta_4 \xi^3) + (\beta_1 + 3\beta_4 \xi^2)x + (\beta_2 - 3\beta_4 \xi)x^2 + (\beta_3 + \beta_4)x^3$$

$$\begin{aligned} f_2(x = \xi) &= (\beta_0 - \beta_4 \xi^3) + (\beta_1 + 3\beta_4 \xi^2)\xi + (\beta_2 - 3\beta_4 \xi)\xi^2 + (\beta_3 + \beta_4)\xi^3 \\ &= \beta_0 - \beta_4 \xi^3 + \beta_1 \xi + 3\beta_4 \xi^3 + \beta_2 \xi^2 - 3\beta_4 \xi^2 + \beta_3 \xi^3 + \beta_4 \xi^3 \\ &= \beta_0 + \beta_1 \xi + \beta_2 \xi^2 + \beta_3 \xi^3 \end{aligned}$$

Therefore, $f_1(x = \xi) = f_2(x = \xi)$.

We now evaluate each of the first derivatives at ξ .

$$f_1(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

$$f_{1'}(x) = \beta_1 + 2\beta_2 x + 3\beta_3 x^2$$

$$f_{1'}(x = \xi) = \beta_1 + 2\beta_2 \xi + 3\beta_3 \xi^2$$

$$f_2(x) = (\beta_0 - \beta_4 \xi^3) + (\beta_1 + 3\beta_4 \xi^2)x + (\beta_2 - 3\beta_4 \xi)x^2 + (\beta_3 + \beta_4)x^3$$

$$f_{2'}(x) = (\beta_1 + 3\beta_4 \xi^2) + 2(\beta_2 - 3\beta_4 \xi)x + 3(\beta_3 + \beta_4)x^2$$

$$\begin{aligned} f_{2'}(x = \xi) &= (\beta_1 + 3\beta_4 \xi^2) + 2(\beta_2 - 3\beta_4 \xi)\xi + 3(\beta_3 + \beta_4)\xi^2 \\ &= \beta_1 + 3\beta_4 \xi^2 + 2\beta_2 \xi - 6\beta_4 \xi^2 + 3\beta_3 \xi^2 + 3\beta_4 \xi^2 \\ &= \beta_1 + 2\beta_2 \xi + 3\beta_4 \xi^2 \end{aligned}$$

Therefore, $f_{1'}(x = \xi) = f_{2'}(x = \xi)$.

We now evaluate each of the second derivatives at ξ .

$$f_{1'}(x) = \beta_1 + 2\beta_2 x + 3\beta_3 x^2$$

$$f_{1''}(x) = 2\beta_2 + 6\beta_3 x$$

$$f_{1''}(x = \xi) = 2\beta_2 + 6\beta_3 \xi$$

$$f_{2'}(x) = (\beta_1 + 3\beta_4 \xi^2) + 2(\beta_2 - 3\beta_4 \xi)x + 3(\beta_3 + \beta_4)x^2$$

$$f_{2''}(x) = 2(\beta_2 - 3\beta_4 \xi) + 6(\beta_3 + \beta_4)x$$

$$\begin{aligned} f_{2''}(x = \xi) &= 2(\beta_2 - 3\beta_4 \xi) + 6(\beta_3 + \beta_4)\xi \\ &= 2\beta_2 + 6\beta_3 \xi \end{aligned}$$

Therefore, $f_{1''}(x = \xi) = f_{2''}(x = \xi)$.

Since $f(x)$, $f'(x)$ and $f''(x)$ are continuous, $f(x)$ is indeed a spline.

Reflection

While completing Question 1 of the tutorial, I was able to carry out many trial-and-error processes while also visualizing the model. I am now confident in basis construction. One of the biggest misunderstandings that I have had during implementation is the difference in ‘indicator’ columns between the discontinuous and continuous regression scenarios. When working in a discontinuous scenario, observations can only belong to 1 segment. In a continuous setting, observations influence more than just the one segment they directly fall in.

Multidimensional Splines

Tutorial 2

```
library(mgcv)
# simulate data from a surface
n <- 5000
eg <- gamSim(2, n = n, scale = 0.5)
```

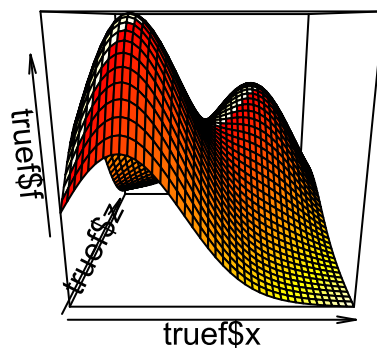
Bivariate smoothing example

```
attach(eg)

truef <- eg$truth

#?persp
persp(truef$x, truef$z, truef$f, col = heat.colors(40), main = "a. truth")
```

a. truth

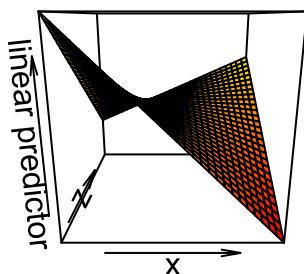


```
?smooth.terms
?s
?smooth.construct.tp.smooth.spec
?te

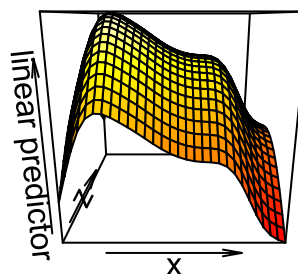
# reproducing plots from tut

par(mfrow = c(1, 2))
b1 <- gam(y ~ x + z + x*z, data = data)
vis.gam(b1, main = "b. linear model")
# additive splines
b2 <- gam(y ~ s(x) + s(z), data = data)
vis.gam(b2, main = "c. additive splines")
```

b. linear model

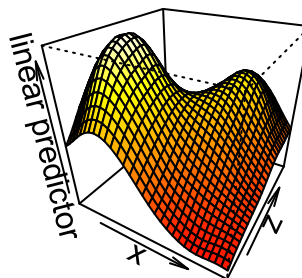
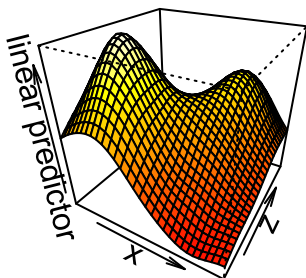


c. additive splines



```
par(mfrow = c(1, 2))
ind200 <- sample(1:n, 200, replace = FALSE)
b5 <- gam(y ~ s(x, z, k = 40), data = data,
knots = list(x = data$x[ind200], z = data$z[ind200]))
vis.gam(b5, theta = 30, phi = 30, main = "low rank thin-plate spline")
## and a pure "knot based" spline of the same data
b6<- gam(y ~ s(x, z, k = 64), data = data,
knots = list(x = rep((1:8-0.5)/8,8),
z = rep((1:8-0.5)/8, rep(8,8))))
vis.gam(b6,color = "heat", theta = 30, phi = 30, main = "knot-based thin-plate
spline")
```

low rank thin-plate spline knot-based thin-plate splir



```
summary(b6)
```

```
Family: gaussian  
Link function: identity
```

```
Formula:  
y ~ s(x, z, k = 64)
```

```
Parametric coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.298286	0.007102	42	<2e-16 ***

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Approximate significance of smooth terms:
```

	edf	Ref.df	F	p-value
s(x,z)	23.77	32.11	17.44	<2e-16 ***

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
R-sq.(adj) =  0.102   Deviance explained = 10.6%
```

```
GCV = 0.25345   Scale est. = 0.25219   n = 5000
```

```
detach(eg)
```

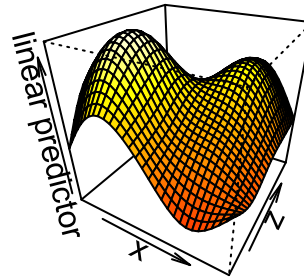
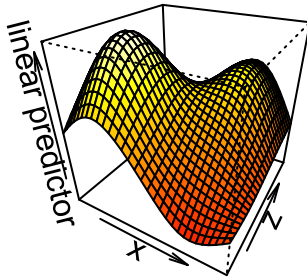
Fit tensor product splines

```
# Questions in tut
attach(eg)
# fit tensor product spline with and without penalty.
par(mfrow = c(1, 2))

# with penalty
tps1 <- gam(y ~ te(x, z, fx = FALSE), data = data)
vis.gam(tps1, color = "heat", theta = 30, phi = 30, main = "tensor product spline
(with penalty)")

# without penalty
tps2 <- gam(y ~ te(x, z, fx = TRUE), data = data)
vis.gam(tps2, color = "heat", theta = 30, phi = 30, main = "tensor product spline
(without penalty)")
```

tensor product spline (with penalty) tensor product spline (without penalty)



```
par(mfrow = c(2, 2))
# without choosing knots
tps3 <- gam(y ~ te(x, z, fx = FALSE), data = data)
vis.gam(tps3, color = "heat", theta = 30, phi = 30, main = "tensor product spline
(knots not specified)")

ind10 <- sample(1:n, 10, replace = FALSE)
```

```

# with choosing knots
tps4 <- gam(y ~ te(x, z, fx = FALSE, k = c(10, 10)), data = data, knots = list(x
= data$x[ind10], z = data$z[ind10]))
vis.gam(tps4,color = "heat", theta = 30, phi = 30, main = "tensor product spline
(10 knots)")

ind20 <- sample(1:n, 20, replace = FALSE)
tps5 <- gam(y ~ te(x, z, fx = FALSE, k = c(20, 20)), data = data, knots = list(x
= data$x[ind20], z = data$z[ind20]))
vis.gam(tps5,color = "heat", theta = 30, phi = 30, main = "tensor product spline
(20 knots)")

ind30 <- sample(1:n, 30, replace = FALSE)
tps6 <- gam(y ~ te(x, z, fx = FALSE, k = c(30, 30)), data = data, knots = list(x
= data$x[ind30], z = data$z[ind30]))
vis.gam(tps6,color = "heat", theta = 30, phi = 30, main = "tensor product spline
(30 knots)")

```

tensor product spline (knots not specified)



tensor product spline (10 knots)



tensor product spline (20 knots)



tensor product spline (30 knots)



```

# changing the smoothing paramters
### *****easier to analyse differences visually when plots not in grid. *****
tps7 <- gam(y ~ te(x, z, fx = FALSE, sp = c(0.01, 0.01)), data = data)
vis.gam(tps7,color = "heat", theta = 30, phi = 30, main = "tensor product spline:
0.01 smoothing")

```

```
tps8 <- gam(y ~ te(x, z, fx = FALSE, sp = c(0.1, 0.1)), data = data)
vis.gam(tps8,color = "heat", theta = 30, phi = 30, main = "tensor product spline:
0.1 smoothing")
```

```
tps9 <- gam(y ~ te(x, z, fx = FALSE, sp = c(1, 1)), data = data)
vis.gam(tps9,color = "heat", theta = 30, phi = 30, main = "tensor product spline:
1 smoothing")
```

```
tps10 <- gam(y ~ te(x, z, fx = FALSE, sp = c(10, 10)), data = data)
vis.gam(tps10,color = "heat", theta = 30, phi = 30, main = "tensor product
spline: 10 smoothing")
```

tensor product spline: 0.01 smooth tensor product spline: 0.1 smooth

linear predictor

linear predictor

tensor product spline: 1 smoothing tensor product spline: 10 smoothing

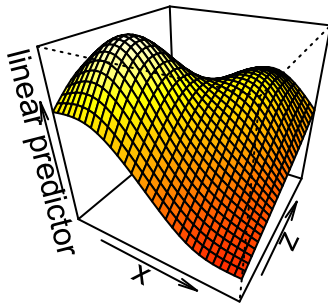
linear predictor

linear predictor

```
par(mfrow = c(1, 1))
```

```
tps11 <- gam(y ~ te(x, z, fx = FALSE, sp = c(100, 100)), data = data)
vis.gam(tps11,color = "heat", theta = 30, phi = 30, main = "tensor product
spline: 100 smoothing")
```

tensor product spline: 100 smoothing



```
detach(eg)
```

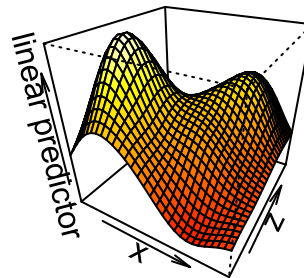
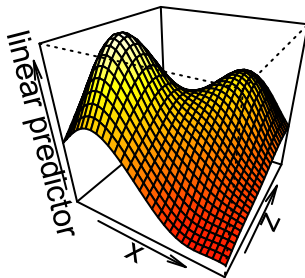
Fit thin plate splines

```
attach(eg)
# fit thin plate spline with and without penalty.
par(mfrow = c(1, 2))

# with penalty
thinps1 <- gam(y ~ s(x, z, fx = FALSE), data = data)
vis.gam(thinps1, color = "heat", theta = 30, phi = 30, main = "thin plate spline
(with penalty)")

# without penalty
thinps2 <- gam(y ~ s(x, z, fx = TRUE), data = data)
vis.gam(thinps2, color = "heat", theta = 30, phi = 30, main = "thin plate spline
(without penalty)")
```

thin plate spline (with penaltthin plate spline (without pen

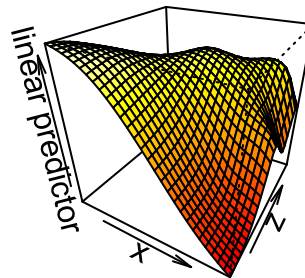
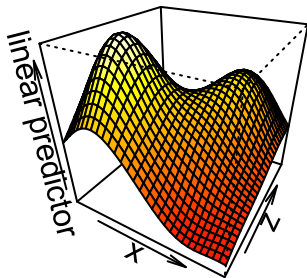


```
#par(mfrow = c(2, 2))
# without choosing knots
thinps3 <- gam(y ~ s(x, z, fx = FALSE), data = data)
vis.gam(thinps3, color = "heat", theta = 30, phi = 30, main = "thin plate spline
(knots not specified)")

ind10 <- sample(1:n, 10, replace = FALSE)

# with choosing knots
thinps4 <- gam(y ~ s(x, z, fx = FALSE, k = 10), data = data, knots = list(x =
data$x[ind10], z = data$z[ind10]))
vis.gam(thinps4, color = "heat", theta = 30, phi = 30, main = "thin plate spline
(10 knots)")
```

in plate spline (knots not spe thin plate spline (10 knots

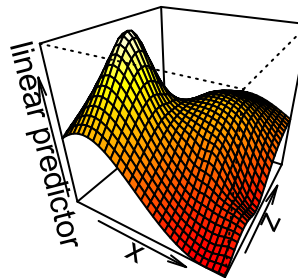
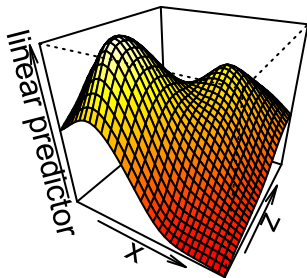


```
ind20 <- sample(1:n, 20, replace = FALSE)
thinps5 <- gam(y ~ s(x, z, fx = FALSE, k = 20), data = data, knots = list(x =
data$x[ind20], z = data$z[ind20]))
vis.gam(thinps5,color = "heat", theta = 30, phi = 30, main = "thin plate spline
(20 knots)")
```

```
ind30 <- sample(1:n, 30, replace = FALSE)
thinps6 <- gam(y ~ s(x, z, fx = FALSE, k = 30), data = data, knots = list(x =
data$x[ind30], z = data$z[ind30]))
vis.gam(thinps6,color = "heat", theta = 30, phi = 30, main = "thin plate spline
(30 knots)")
```

thin plate spline (20 knots

thin plate spline (30 knots

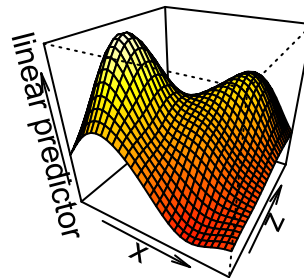
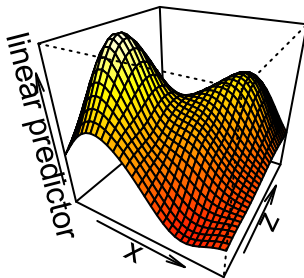


```
# changing the smoothing parameters
### *****easier to analyse differences visually when plots not in grid.
*****

things7 <- gam(y ~ s(x, z, fx = FALSE, sp = 0.01), data = data)
vis.gam(things7,color = "heat", theta = 30, phi = 30, main = "thin plate spline:
0.01 smoothing")

things8 <- gam(y ~ s(x, z, fx = FALSE, sp = 0.1), data = data)
vis.gam(things8,color = "heat", theta = 30, phi = 30, main = "thin plate spline:
0.1 smoothing")
```

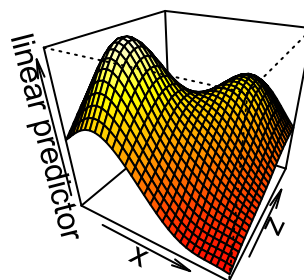
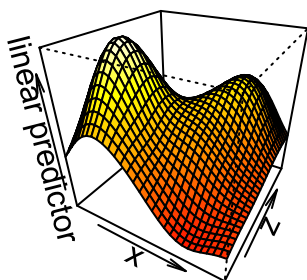
thin plate spline: 0.01 smooth thin plate spline: 0.1 smooth



```
thinps9 <- gam(y ~ s(x, z, fx = FALSE, sp = 1), data = data)
vis.gam(thinps9, color = "heat", theta = 30, phi = 30, main = "thin plate spline:
1 smoothing")
```

```
thinps10 <- gam(y ~ s(x, z, fx = FALSE, sp = 10), data = data)
vis.gam(thinps10, color = "heat", theta = 30, phi = 30, main = "thin plate spline:
10 smoothing")
```

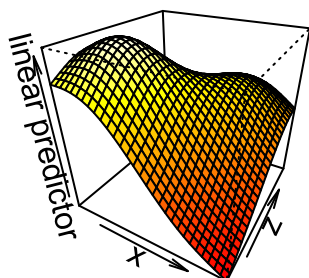
thin plate spline: 1 smoothi thin plate spline: 10 smooth



```
thinps11 <- gam(y ~ s(x, z, fx = FALSE, sp = 100), data = data)
vis.gam(thinps11, color = "heat", theta = 30, phi = 30, main = "thin plate spline:
100 smoothing")
```

```
detach(eg)
```

thin plate spline: 100 smooth



Fit tensor product splines with custom function

```
# function to construct tensor product basis + fit to data
# construct individual basis columns for each variable first
# multiply (cross product) of the columns of each var - tensor product part
# accommodate for knots
# fit to data
# ?bs() ## constructs the initial basis matrix for 1 var

tensor_basis <- function(x_data_matrix, knots){
  # x must be a data_matrix
  # nknots should be a vector of values - 1 value for each variable
  # knots should be a list of vectors with the knots for each variable
  # y is a vec of the response values

  # construct individual basis matrices
  x1 <- x_data_matrix[,1]
  x2 <- x_data_matrix[,2]

  x1_bmat <- bs(x1, knots = knots[[1]])
  x2_bmat <- bs(x2, knots = knots[[2]])

  # construct product columns

  tensor_mat <- matrix(0, nrow = nrow(x_data_matrix), ncol = ncol(x1_bmat) *
ncol(x2_bmat))
  column <- 1
```

```

for ( i in 1:ncol(x1_bmat)) {
  for ( j in 1:ncol(x2_bmat)) {

    tensor_mat[,column] <- x1_bmat[,i] * x2_bmat[,j]
    column = column + 1

  }
}

#beta_hat <- solve(t(tensor_mat) %*% tensor_mat) %*% t(tensor_mat) %*% y
#y_hat <- tensor_mat %*% beta_hat

#return(list(t_basis = tensor_mat, betas = beta_hat, fitted = y_hat))
return(list(t_basis = tensor_mat))
}

attach(eg)

x_data_matrix <- cbind(data$x, data$z)
y <- data$y

set.seed(8)
ind5 <- sample(1:nrow(x_data_matrix), 5, replace = F)

x_knots <- data$x[ind5]
z_knots <- data$z[ind5]

data_basis <- tensor_basis(x_data_matrix = x_data_matrix, knots = list(x_knots,
z_knots))
data_basis$t_basis <- as.matrix(data_basis$t_basis)

# fitting and prediction
beta_hat <- solve(t(data_basis$t_basis) %*% data_basis$t_basis) %*%
t(data_basis$t_basis) %*% y
y_hat <- data_basis$t_basis %*% beta_hat

x_grid_vals <- seq(min(data$x), max(data$x), length.out = 50)
z_grid_vals <- seq(min(data$z), max(data$z), length.out = 50)

x_grid_bmat <- bs(x_grid_vals, knots = x_knots)
z_grid_bmat <- bs(z_grid_vals, knots = z_knots)

```

```

pred_grid <- expand.grid(x = x_grid_vals, z = z_grid_vals)

X_grid <- tensor_basis(as.matrix(pred_grid), knots = list(x_knots, z_knots))

fitted_surface <- X_grid$t_basis %*% beta_hat

fitted_surface_matrix <- matrix(fitted_surface, nrow = length(x_grid_vals), ncol
= length(z_grid_vals))

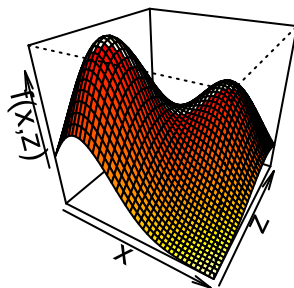
par(mfrow = c(1, 2))

persp(truef$x, truef$z, truef$f, theta = 30, phi = 30, col = heat.colors(40),
xlab = "x", ylab = "z", zlab = "f(x,z)", main = "True surface")

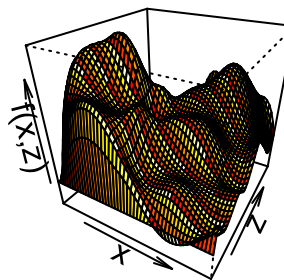
persp(x_grid_vals, z_grid_vals, fitted_surface_matrix, theta = 30, phi = 30,
col = heat.colors(40), xlab = "x", ylab = "z", zlab = "f(x,z)", main = "Fitted
tensor-product spline")

```

True surface



Fitted tensor-product splir



```

par(mfrow = c(1, 1))

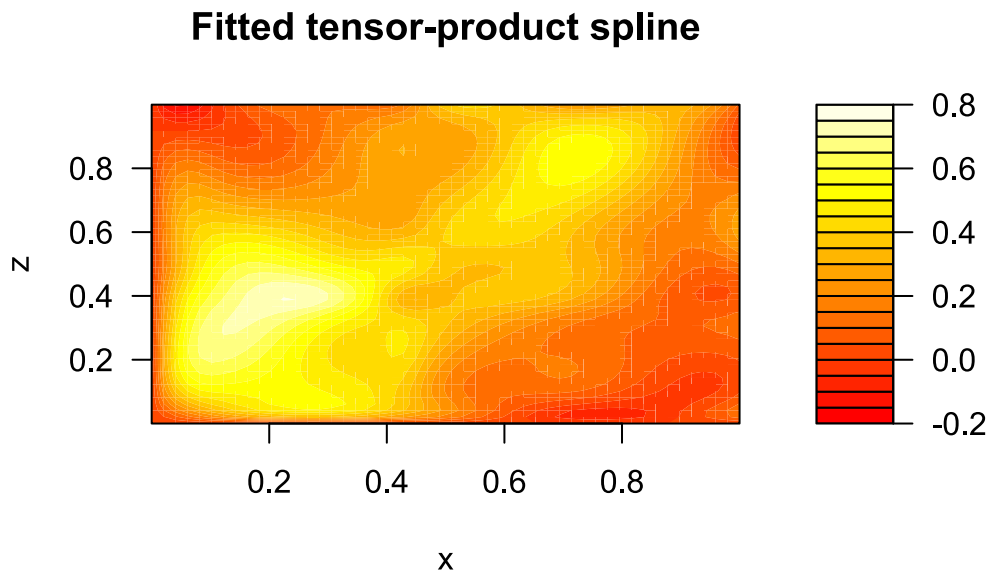
# alternate visualization - this one not striped like 3D plot, indicates possible

```

```

visualization issue with 3D plot
filled.contour( x_grid_vals, z_grid_vals, fitted_surface_matrix, color.palette
= heat.colors, xlab = "x", ylab = "z", main = "Fitted tensor-product spline"
)

```



```
detach(eg)
```

```

### Need to fit with different number of knots (maybe 3, 5 and 7). Larger number
of knots are computationally/numerically expensive

```

Fit thin plate splines with custom function

```

# construct thin plate spline basis + fit to data
## radial basis functions
# calc euclidean dist between obs and knots
# multiply dist^2 by log of dist

thin_plate_basis <- function(x_data_matrix, knots) {
  # x_data_matrix should be a matrix as before
  # knots should also be a matrix where columns represent the variables and rows
  are knots

  x1 <- x_data_matrix[, 1]
  x2 <- x_data_matrix[, 2]

```

```

# Polynomial part of basis mat
thin_p_basis_mat <- cbind(intercept = 1, x1 = x1, x2 = x2, interaction = x1 *
x2)

# Add one radial basis column per knot
for (i in 1:nrow(knots)) {

  # calculate euclidean distance
  r <- sqrt((x1 - knots[i, 1])^2 + (x2 - knots[i, 2])^2)

  phi <- ifelse(r == 0, 0, r^2 * log(r)) # accommodate for when obs is the
same as the knott

  thin_p_basis_mat <- cbind(thin_p_basis_mat, phi)
}

return(thin_p_basis_mat)
}

# function for optimal coefficients, predictions and plotting
fit_plot_tps <- function(n_knots) {

  grid_n <- sqrt(n_knots)

  # nknots must be a perfect square for both methods to be able to be fitted
with the exact same number of knots
  if (grid_n != floor(grid_n)) {
    stop(
      "n_knots must be a perfect square, e.g. 9, 16, 25, 36."
    )
  }

  grid_n <- as.integer(grid_n)

  # new predictor values to predict on
thin_p_pred_grid <- expand.grid(x = x_grid_vals, z = z_grid_vals) |> as.matrix()

```

```

## Random knots --##--##--##

# get n_knot indices for subset of knots
ind_random <- sample(seq_len(nrow(data)),n_knots)

# define knots as subset according to indices
thin_p_knots_random <- cbind( data$x[ind_random], data$z[ind_random])

# Construct basis
thin_p_bmat_random <- thin_plate_basis(x_data_matrix, thin_p_knots_random)

# get beta hats
thin_p_beta_hat_random <- solve(t(thin_p_bmat_random) %*% thin_p_bmat_random)
%*% t(thin_p_bmat_random) %*% y

# calc basis for predictor values used for prediction
thin_p_X_grid_random <- thin_plate_basis(thin_p_pred_grid,
thin_p_knots_random)

# Prediction surface
thin_p_fitted_surface_random <- thin_p_X_grid_random %*%
thin_p_beta_hat_random

thin_p_fitted_surface_matrix_random <- matrix(thin_p_fitted_surface_random,
nrow = length(x_grid_vals),ncol = length(z_grid_vals))

# Sequential knots --##--##--##

x_knot_vals <- seq( min(data$x), max(data$x), length.out = grid_n)

z_knot_vals <- seq( min(data$z), max(data$z), length.out = grid_n)

thin_p_knots_sequenced <- expand.grid( x = x_knot_vals, z = z_knot_vals) |>
as.matrix()

# Basis matrix
thin_p_bmat_seq <- thin_plate_basis(x_data_matrix, thin_p_knots_sequenced)

```

```

# Estimate new beta hats based on different knots
thin_p_beta_hat_seq <- solve( t(thin_p_bmat_seq) %*% thin_p_bmat_seq) %*%
t(thin_p_bmat_seq) %*%y

# Prediction basis
thin_p_X_grid_seq <- thin_plate_basis(thin_p_pred_grid,thin_p_knots_sequenced)

# Predictions
thin_p_fitted_surface_seq <- thin_p_X_grid_seq %*% thin_p_beta_hat_seq

thin_p_fitted_surface_matrix_seq <- matrix(thin_p_fitted_surface_seq, nrow =
length(x_grid_vals), ncol = length(z_grid_vals))

par(mfrow = c(1, 2))

# -----# Random knots #-#-----##

image(x_grid_vals,z_grid_vals, thin_p_fitted_surface_matrix_random, col =
heat.colors(40), xlab = "x", ylab = "z", main = paste("Random knots: k =",
n_knots))

contour( x_grid_vals, z_grid_vals, thin_p_fitted_surface_matrix_random, add =
TRUE, drawlabels = FALSE)

points(thin_p_knots_random[, 1], thin_p_knots_random[, 2], pch = 19, col =
"blue", cex = 1.2)

# -----# Equally spaced knots -#-----#

image(x_grid_vals,z_grid_vals, thin_p_fitted_surface_matrix_seq, col =
heat.colors(40), xlab = "x", ylab = "z", main = paste( "Sequential knots: k =",
n_knots))

contour(x_grid_vals,z_grid_vals,thin_p_fitted_surface_matrix_seq,add =
TRUE,drawlabels = FALSE)

points( thin_p_knots_sequenced[, 1],thin_p_knots_sequenced[, 2], pch = 19, col
= "blue", cex = 1.2)

#par(mfrow = c(1, 1))

```

```

par(mfrow = c(1, 2))

# --Random knots ----

persp( x_grid_vals, z_grid_vals, thin_p_fitted_surface_matrix_random, theta =
30, phi = 30,col = heat.colors(40), xlab = "x", ylab = "z",  zlab = "f(x,z)",
main = paste(n_knots,
             "random knots" ))

# Equally spaced knots -----

persp(x_grid_vals, z_grid_vals, thin_p_fitted_surface_matrix_seq, theta = 30,
phi = 30, col = heat.colors(40), xlab = "x",ylab = "z", zlab = "f(x,z)", main =
paste( n_knots, "sequential knots" ))

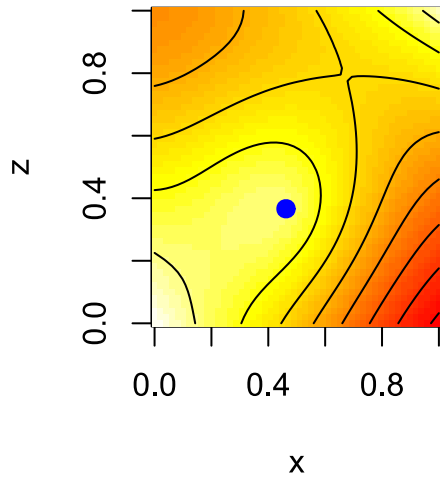
# Reset plotting window
par(mfrow = c(1, 1))
}

attach(eg)

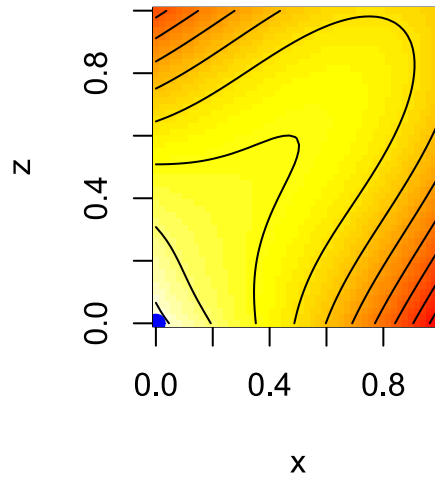
set.seed(8)
for (i in c(1,4,9,16,25)){
  fit_plot_tps(i)
}

```

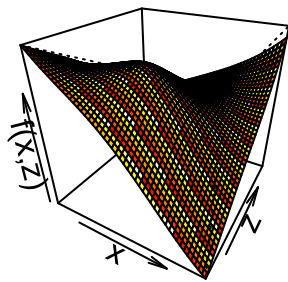
Random knots: $k = 1$



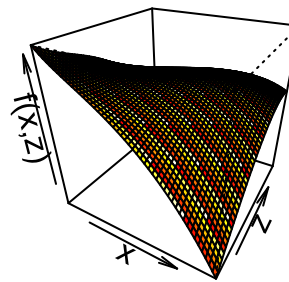
Sequential knots: $k = 1$



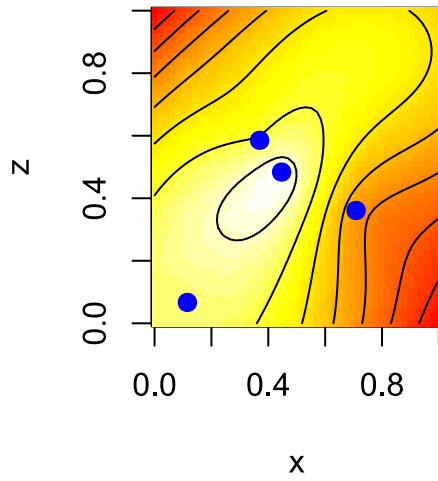
1 random knots



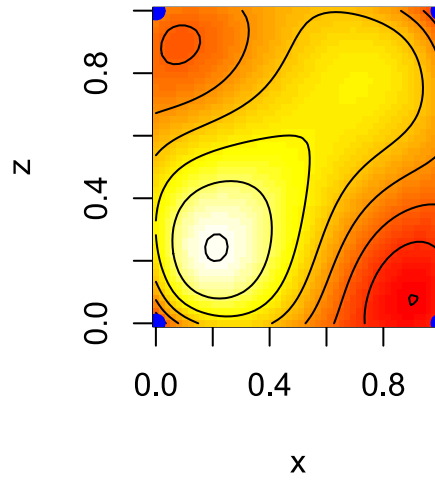
1 sequential knots



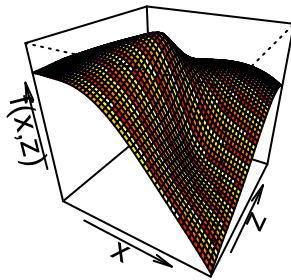
Random knots: $k = 4$



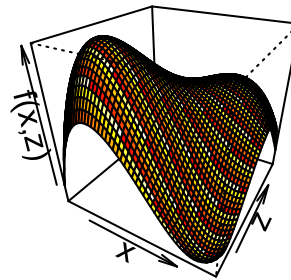
Sequential knots: $k = 4$



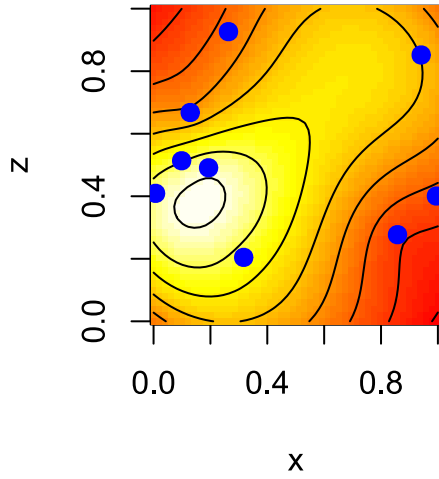
4 random knots



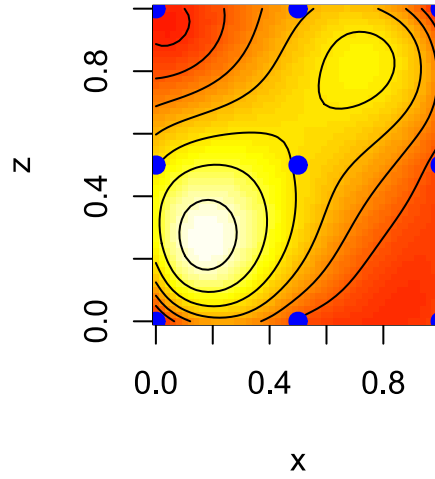
4 sequential knots



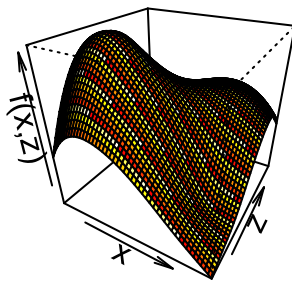
Random knots: $k = 9$



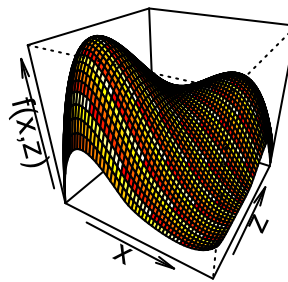
Sequential knots: $k = 9$



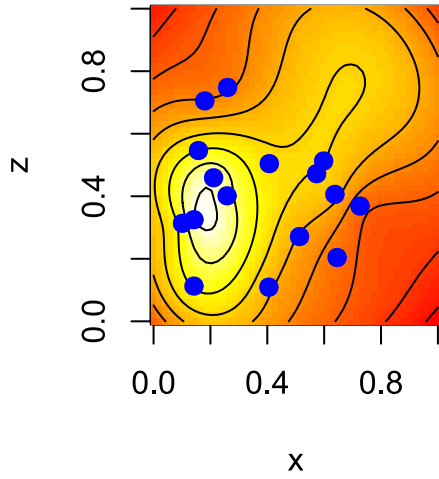
9 random knots



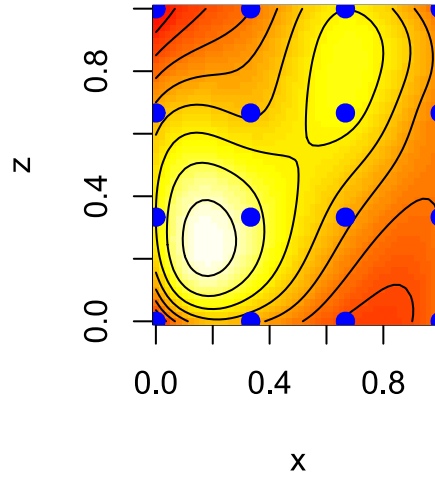
9 sequential knots



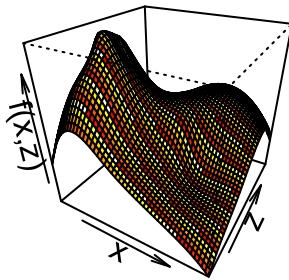
Random knots: $k = 16$



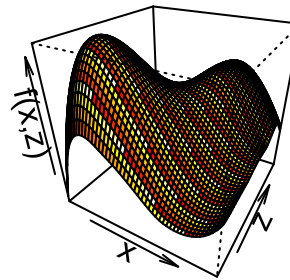
Sequential knots: $k = 16$



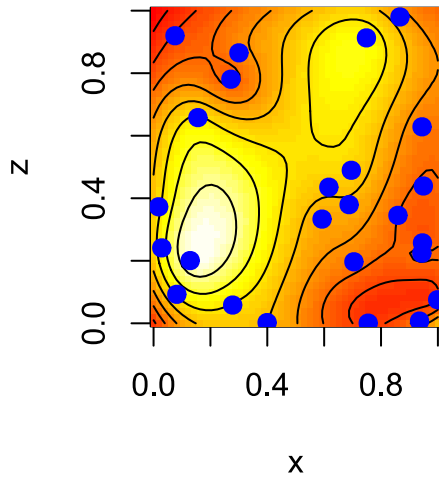
16 random knots



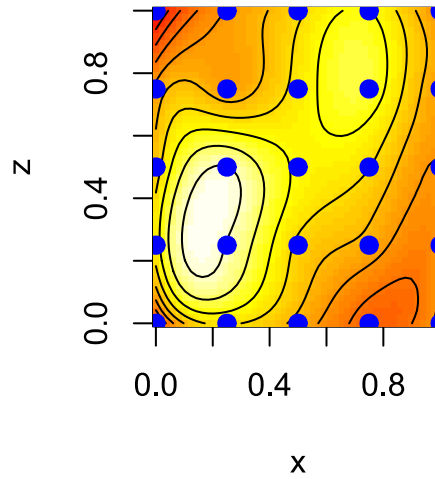
16 sequential knots



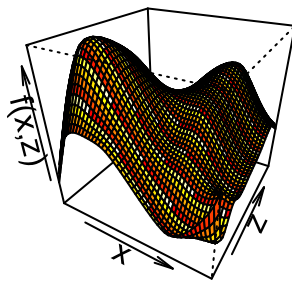
Random knots: $k = 25$



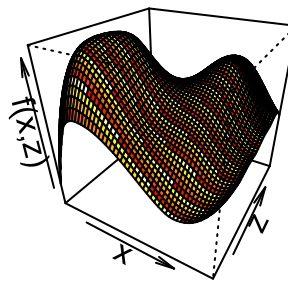
Sequential knots: $k = 25$



25 random knots



25 sequential knots



```
detach(eg)
```

```
# check goodness of fit *****
```

Exercise 1

Show that the β minimizing $\|y - X\beta\|^2 + \lambda\beta'P\beta$ is given by $\beta = (X'X)^{-1}X'y$.

First, expand all terms.

$$\begin{aligned} f(\beta) &= \|y - X\beta\|^2 + \lambda\beta'P\beta \\ &= (y - X\beta)'(y - X\beta) + \lambda\beta'P\beta \\ &= y'y - y'X\beta - \beta'X'y + \beta'X'X\beta + \lambda\beta'P\beta \\ &= y'y - 2y'X\beta + \beta'X'X\beta + \lambda\beta'P\beta \end{aligned}$$

Secondly, find the derivative.

$$f'(\beta) = -2y'X + 2X'X\beta + 2\lambda P\beta$$

Set $f'(\beta) = 0$.

$$\begin{aligned} f'(\beta) &= -2y'X + 2X'X\beta + 2\lambda P\beta = 0 \\ -y'X + X'X\beta + \lambda P\beta &= 0 \\ -y'X + (X'X + \lambda P)\beta &= 0 \\ (X'X + \lambda P)\beta &= X'y \\ (X'X + \lambda P)^{-1}(X'X + \lambda P)\beta &= (X'X + \lambda P)^{-1}X'y \\ \therefore \beta &= (X'X + \lambda P)^{-1}X'y \end{aligned}$$

Gams

Mini project

Summary

Summary

This study investigated the sensitivity of generalized additive models (GAMs) to the choice of basis dimension under different sample size conditions. Simulated Walleye data were generated for sample sizes of 50, 150 and 500, with nonlinear relationships between age, length and weight. GAMs were then fitted using basis dimensions of $k=3$, $k=5$, and $k=10$, with a linear regression model included as a baseline comparison. The same analysis was subsequently applied to the WalleyeErie2 real-world data set.

Increasing the basis dimension generally resulted in higher effective degrees of freedom (EDF), particularly for the length effect, indicating that greater basis dimensions allowed more complex smooth functions to be estimated. However, changes in adjusted R^2 , deviance explained and scale

were relatively small. This indicates that increasing the maximum available flexibility did not necessarily produce substantial improvements in overall model fit. Larger sample sizes allowed more complex smooths to be estimated, suggesting that sample size influences the amount of flexibility that can be supported by the data.

The linear models performed relatively well for the simulated data despite the nonlinear data-generating relationships, demonstrating that linear models can provide reasonable approximations to nonlinear relationships over a restricted range. The GAMs provided modest improvements in model fit. For the real-world Walleye data, nonlinear effects were consistently identified for both length and weight, although increasing the basis dimension again resulted in only small improvements in model-level fit measures.

Overall, the findings demonstrate that basis dimension controls the maximum flexibility available to a GAM but does not directly determine the complexity of the final fitted model. The results also highlight the importance of considering EDF and diagnostic results alongside model fit measures when assessing basis dimension. However, because only one simulated data set was generated for each sample size and predictive performance was not assessed, further simulation repetitions and out-of-sample validation would be required to draw more general conclusions about basis-dimension sensitivity.

Reflection

I decided on this topic since it was fairly broad and allowed me to explore the algorithm and application. I decided on the specific data sizes since there are many data sets that are within these ranges. Furthermore, I enjoy the ecological field. Hence, I preferred to explore a biological relationship. I was finding difficulty finding ecological data sets online that was applicable to this study. After reading about this FSA package, I decided I would apply the methods to one of these.

With regards to the results of this study, it is not what I expected. I did hypothesize that model fit would improve. However, it was only flexibility that showed considerable change for many models

MARS

Mars tutorial Q1: earth package exploration

Numerous models were fit, each with different penalty values. To compare the effect of this, I looked at the model performance metrics. This is summarized in Table 1. As degree increases, there is a steady decline in the GCV error (from 0.075 to 0.051) and residual sum of squares (from 340 to 228). These indicate lower prediction error and greater fit. The R^2 and generalized R^2 are basically the same values and increases with degree (from 0.69 to 0.79), indicating that a further 10% of variation in the response was explained by the additional terms being allowed into the model when 4th order interactions are allowed. And as expected, the number of terms and predictors increase with more terms and relationships being allowed. Although increasing the degree results in an overall better predictive model, it is a less parsimonious one.

```

# exploration of degree Rsq continues increasing
spam.earth1 <- earth(spambase$spam_or_not ~., data = spambase, degree = 1,
glm=list(family=binomial))
summary(spam.earth1, digits = 2, style = 'bf')

# to get p values for coefficients
spam.earth1_bm <- spam.earth1$bx
spam.earth1_glm <- glm(spambase$spam_or_not ~ spam.earth1_bm[, -1], family =
binomial)

summary(spam.earth1_glm)

spam.earth2 <- earth(spambase$spam_or_not ~., data = spambase, degree = 2,
glm=list(family=binomial))
summary(spam.earth2, digits = 2, style = 'bf')

spam.earth3 <- earth(spambase$spam_or_not ~., data = spambase, degree = 3,
glm=list(family=binomial))
summary(spam.earth3, digits = 2, style = 'bf')

spam.earth4 <- earth(spambase$spam_or_not ~., data = spambase, degree = 4,
glm=list(family=binomial))
summary(spam.earth4, digits = 2, style = 'bf')

```

Table 1: Model performance metrics and number of terms in mode after varying the degree of the MARS model.

de- gree	GCV	RSS	GRsq	RSq	terms_se- lected	terms_to- tal	predictors_se- lected	predictors_to- tal
1	0.075	340	0.69	0.69	16	21	10	57
2	0.063	281	0.74	0.74	23	29	12	57
3	0.057	256	0.76	0.77	27	37	15	57
4	0.051	228	0.79	0.79	31	52	22	57

When investigating the effect of changing the penalty, the degree was kept constant at 2. There are 3 different scenarios that were tested (Table 2). Firstly, the penalty value of -1 implies that there was no penalty is used, and this equates to the GCV error just being the mean RSS. A penalty value of 0 means that only terms are penalized (not individual knots). Penalties greater than 0 implement penalties per knot of that value. Here, penalty values of 3 and 4 were tested. There is no change across these different scenarios, except for the GCV error increasing by 0.001 as term-based penalties are used, and then by another 0.001 when penalty 3 was used. This suggests that the additional terms retained under lower penalties provide only marginal improvements in fit and that the main predictive structure of the model is relatively robust to the choice of penalty.

```
# exploration of penalty
degree = 2
spam.earth_p1 <- earth(spambase$spam_or_not ~., data = spambase, degree = degree,
penalty = if(degree > 1) 3 else 2, , glm=list(family=binomial)) # default
penalty
summary(spam.earth_p1, digits = 2, style = 'bf')

spam.earth_p2 <- earth(spambase$spam_or_not ~., data = spambase, degree = degree,
penalty = 4, glm=list(family=binomial))
summary(spam.earth_p2, digits = 2, style = 'bf')

spam.earth_p3 <- earth(spambase$spam_or_not ~., data = spambase, degree = degree,
penalty = 0, glm=list(family=binomial)) # penalty = 0 penalises only terms not
knots
summary(spam.earth_p3, digits = 2, style = 'bf')

spam.earth_p4 <- earth(spambase$spam_or_not ~., data = spambase, degree = degree,
penalty = -1, glm=list(family=binomial)) # penalty = -1 implies no penalty
summary(spam.earth_p4, digits = 2, style = 'bf')
```

Table 2: Model performance metrics and number of terms in mode after varying the penalty of the MARS model.

penalty	GCV	RSS	GRSq	RSq	terms_se- lected	terms_to- tal	predictors_se- lected	predictors_to- tal
3	0.063	281	0.74	0.74	23	29	12	57
4	0.063	283	0.74	0.74	18	29	12	57
0	0.062	281	0.74	0.74	26	29	12	57
-1	0.061	280	0.74	0.74	29	29	12	57

The parameter *nk* controls the maximum number of terms allowed in the model before pruning begins. This includes the intercept. Table 3 displays the results of changing this parameter. Increasing *nk* to 100 has no effect, because the model only requires 29 terms before pruning. However, restricting *nk* to 20 and 10 substantially reduces model complexity and worsens model fit. In particular, RSq/GRSq falls from 0.74 to 0.71 and then 0.61, while GCV increases from 0.063 to 0.070 and 0.094.

This suggests that an *nk* value below approximately 20 restricts the model too severely, preventing Earth from generating enough candidate terms to capture the relationships in the data.

```
# exploration of nk - this is the maximum number of terms before pruning
spam.earth_nk1 <- earth(spambase$spam_or_not ~., data = spambase, degree = 2,
```

```

glm=list(family=binomial)) #default (nk)
summary(spam.earth_nk1, digits = 2, style = 'bf')

# restricting the max no of terms to a number greater than the total amount of
terms will not affect the model
spam.earth_nk2 <- earth(spambase$spam_or_not ~., data = spambase, degree = 2,
nk = 100, glm=list(family=binomial))
summary(spam.earth_nk2, digits = 2, style = 'bf')

# limiting number of terms reduces the model fit (Rsqr)
spam.earth_nk3 <- earth(spambase$spam_or_not ~., data = spambase, degree = 2,
nk = 20, glm=list(family=binomial))
summary(spam.earth_nk3, digits = 2, style = 'bf')

spam.earth_nk4 <- earth(spambase$spam_or_not ~., data = spambase, degree = 2,
nk = 10, glm=list(family=binomial)) #
summary(spam.earth_nk4, digits = 2, style = 'bf')

```

Table 3: Model performance metrics and number of terms in model after varying the maximum number of allowed terms in model before pruning of the MARS model.

nk	GCV	RSS	GRsq	RSq	terms_selected	terms_total	predictors_selected	predictors_total
100	0.063	281	0.74	0.74	23	29	12	57
20	0.070	318	0.71	0.71	14	19	9	57
10	0.094	431	0.61	0.61	7	9	4	57

The thresh parameter controls the threshold for the minimum increase in R^2 for a term to be included in model. The results show that thresh has little effect when set to small values, as the models for 0.001, 0.0001, and 0.00001 are identical. However, increasing the threshold substantially reduces the number of terms selected. At thresh = 0.1, only 3 of 5 candidate terms are retained, resulting in a decrease in RSq/GRsq from 0.74 to 0.49 and an improvement in GCV from 0.063 to 0.12. At thresh = 0.99, no predictors are selected and the model reduces to an intercept-only model, with RSq and GRsq equal to 0.

Thus, larger threshold values cause Earth to terminate the forward selection process earlier, producing simpler but substantially poorer-fitting models. Conversely, reducing the threshold below 0.001 does not improve the model because the same 29 candidate terms are generated and the same 23 terms are ultimately retained.

```

# exploration of thresh

spam.earth_thresh1 <- earth(spambase$spam_or_not ~., data = spambase, degree =
2, thresh = 0.001, glm=list(family=binomial))

```

```
summary(spam.earth_thresh1, digits = 2, style = 'bf')

# increasing the thresh - reduces fit drastically
spam.earth_thresh2 <- earth(spambase$spam_or_not ~., data = spambase, degree =
2, thresh = 0.1, glm=list(family=binomial))
summary(spam.earth_thresh2, digits = 2, style = 'bf')

spam.earth_thresh3 <- earth(spambase$spam_or_not ~., data = spambase, degree =
2, thresh = 0.99, glm=list(family=binomial))
summary(spam.earth_thresh3, digits = 2, style = 'bf')

# decreasing the thresh value - does not improve mod fit
spam.earth_thresh4 <- earth(spambase$spam_or_not ~., data = spambase, degree =
2, thresh = 0.0001, glm=list(family=binomial))
summary(spam.earth_thresh4, digits = 2, style = 'bf')

spam.earth_thresh5 <- earth(spambase$spam_or_not ~., data = spambase, degree =
2, thresh = 0.00001, glm=list(family=binomial))
summary(spam.earth_thresh5, digits = 2, style = 'bf')
```

thresh	GCV	RSS	GRsq	RSq	terms_se- lected	terms_to- tal	predictors_se- lected	predictors_to- tal
0.00001	0.063	281	0.74	0.74	23	29	12	57
0.00010	0.063	281	0.74	0.74	23	29	12	57
0.00100	0.063	281	0.74	0.74	23	29	12	57
0.10000	0.120	564	0.49	0.49	3	5	2	57
0.99000	0.240	1099	0.00	0.00	1	1	0	57

The pruning method is determined by the *pmethod* argument. the default is backward selection. This is performed after the forward selection step to determine how many terms should remain in the model. This is done by selecting the number of terms that results in the highest average test R^2 value. One can also retain all the terms from the forward selection using *pmethod* = 'none'.

The following is output from an earlier model (`earth(spambase$spam_or_not ~., data = spambase, degree = degree, penalty = 4, glm=list(family=binomial))`), The degree 2 MARS model contains 23 selected basis terms, including 4 main-effect terms and 18 two-way interaction terms. The model therefore allows the effect of one predictor to depend on the value of another predictor. For example, terms such as `bf5 * bf1` and `bf6 * bf4` represent interactions between the corresponding hinge functions of V5 and V25, and V7 and V53, respectively. The coefficients determine the direction and magnitude of each basis function's contribution to the log-odds of an observation being

spam. Positive coefficients increase the log-odds of spam when the associated basis functions are active, while negative coefficients decrease them. The relatively large coefficients for some interaction terms indicate that certain combinations of predictor values have a substantial effect on the predicted log-odds. Overall, the model is more complex than the degree 1 model because it incorporates interactions between predictors rather than modelling their effects independently.

```
Call: earth(formula=spambase$spam_or_not~., data=spambase,
            glm=list(family=binomial), degree=2)
```

```
GLM `spambase$spam_or_not` =
```

```
-0.75
-      16 * bf1
-    0.93 * bf2
-     2.9 * bf3
+     36 * bf4
-     6.9 * bf5 * bf1
-    150 * bf6 * bf4
+     4.7 * bf7 * bf4
-     24 * bf8 * bf4
+     7.7 * bf9 * bf4
-     1.5 * bf1 * bf10
+     84 * bf1 * bf11
+     1.3 * bf1 * bf12
+     18 * bf1 * bf13
+     6.3 * bf1 * bf14
+     14 * bf1 * bf15
-     9.4 * bf16 * bf4
+     5.7 * bf17 * bf4
-     0.41 * bf1 * bf18
+ 0.00054 * bf1 * bf19
- 0.0077 * bf1 * bf20
-      18 * bf21 * bf4
-     2.2 * bf22 * bf4
```

```
bf1  h(0.52-V25)
bf2  h(V25-0.52)
bf3  h(0.246-V52)
bf4  h(0.09-V53)
bf5  h(0.43-V5)
bf6  h(0.19-V7)
bf7  h(V7-0.19)
bf8  h(0.32-V16)
bf9  h(V16-0.32)
bf10 h(V27-0.08)
bf11 h(0.08-V27)
bf12 h(V37-0.23)
bf13 h(0.23-V37)
bf14 h(1.81-V42)
```

```

bf15 h(0.4-V46)
bf16 h(0.69-V25)
bf17 h(V25-0.69)
bf18 h(13-V56)
bf19 h(V57-226)
bf20 h(226-V57)
bf21 h(1.777-V52)
bf22 h(V52-1.777)

GLM (family binomial, link logit):
nulldev df      dev  df   devratio      AIC iters converged
  6200 4600      1600 4578         0.74    1660    13         1

Earth selected 23 of 29 terms, and 12 of 57 predictors
Termination condition: RSq changed by less than 0.001 at 29 terms
Importance: V7, V53, V25, V46, V56, V27, V52, V16, V42, V5, V37, V57, ...
Number of terms at each degree of interaction: 1 4 18
Earth GCV 0.063    RSS 281    GRSq 0.74    RSq 0.74

```

Mars tutorial Q2: function for basis function construction

The function `basis()` constructs the two hinge basis functions used in a MARS model for a single predictor X and a specified knot. The first function, $h_1(x) = \max(0, x - \text{knot})$, is zero when x is below the knot and increases linearly when x exceeds the knot. The second function, $h_2(x) = \max(0, \text{knot} - x)$, behaves in the opposite direction: it decreases linearly as x approaches the knot from below and is zero above the knot.

In this example, a knot of 5 is used and X ranges from 1 to 10. The plots illustrate the two complementary hinge functions, which form the basic building blocks used by MARS to model nonlinear relationships.

```

basis <- function(x, knot){

  h1 <- pmax(x - knot, 0)
  h2 <- pmax(knot - x, 0)

  return(list(h1 = h1, h2 = h2))
}

# demonstration - knot at 5
x <- 1:10

b <- basis(x, 5)
h1 <- b$h1
h2 <- b$h2

par(mfrow = c(1,2))

```

```
plot(x, h1, type = 'l')
plot(x, h2, type = 'l')
```

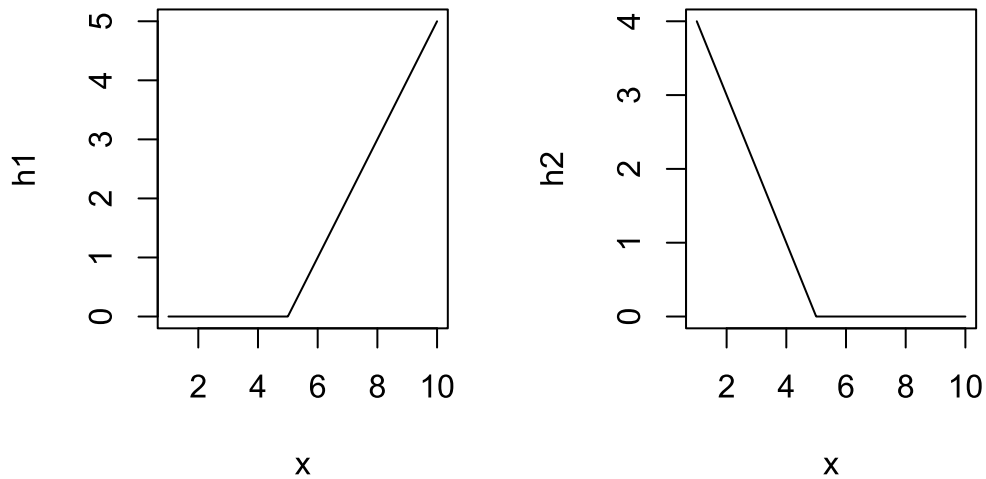


Figure 1: Hinge functions created by custom function

Mars - analysis of swiss data (numeric Y)

The following swiss data set was analyzed using MARS. The degree of allowed interaction was varied amongst the models. Infant mortality was predicted using a range of possible variables, example fertility. The response is continuous.

```
data(swiss)

swiss.earth1 <- earth(Infant.Mortality ~., data = swiss, degree = 1)
plotmo(swiss.earth1)
summary(swiss.earth1, digits = 2)
summary(swiss.earth1, digits = 2, style = 'bf')

swiss.earth2 <- earth(Infant.Mortality ~., data = swiss, degree = 2)
plotmo(swiss.earth2)
summary(swiss.earth2, digits = 2)
summary(swiss.earth2, digits = 2, style = 'bf')

# from hereon, Rsq does not change
swiss.earth3 <- earth(Infant.Mortality ~., data = swiss, degree = 3)
plotmo(swiss.earth3)
```

```
summary(swiss.earth3, digits = 2)
summary(swiss.earth3, digits = 2, style = 'bf')
```

The best performing model according to GCV error, RSS and generalized R^2 was the model of degree 1. This model is an additive model containing 6 basis-function terms, using 4 of the 5 available predictors (agriculture was excluded as hypothesized). The coefficient of -0.41 for $h(68.9\text{-Fertility})$ indicates that, when Fertility is below 68.9, the corresponding basis function contributes negatively to infant mortality. Similarly, the positive coefficient for $h(\text{Education}-13)$ indicates an increasing contribution to predicted infant mortality when Education exceeds 13. Examination is represented by two opposing hinge functions around a knot of 15, allowing its relationship with infant mortality to change on either side of this threshold. Catholic is also represented by two hinge functions, with a change in behaviour around 91.38. Overall, the model captures nonlinear relationships through these hinge functions while remaining additive, as no interaction terms are included. The model has an R^2 of 0.65. This means that approximately 65% of the variation in infant mortality is explained by the fitted model.

```
plotmo grid:   Fertility Agriculture Examination Education Catholic
                70.4         54.1          16          8    15.14
```

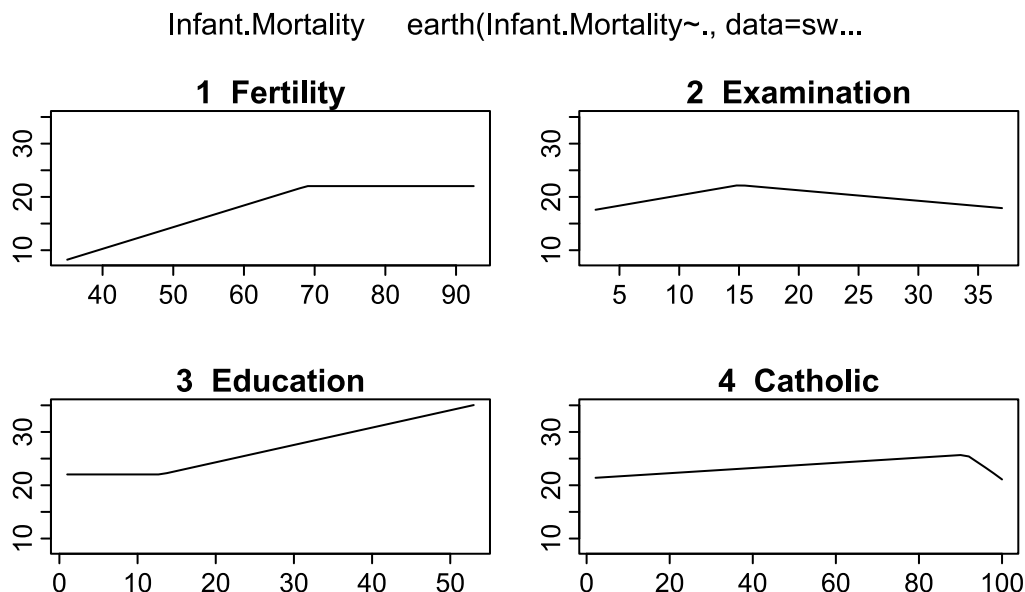


Figure 2: Plots showing the change in infant mortality for each predictor. Infant mortality increases till a fertility rating of approximately 70, after which it remains constant

```
Call: earth(formula=Infant.Mortality~., data=swiss, degree=1)
```

```

Infant.Mortality =
  26
- 0.41 * bf1
- 0.39 * bf2
- 0.2 * bf3
+ 0.33 * bf4
- 0.57 * bf5
- 0.049 * bf6

bf1 h(68.9-Fertility)
bf2 h(15-Examination)
bf3 h(Examination-15)
bf4 h(Education-13)
bf5 h(Catholic-91.38)
bf6 h(97.16-Catholic)

Selected 7 of 16 terms, and 4 of 5 predictors
Termination condition: Reached nk 21
Importance: Catholic, Fertility, Examination, Education, ...
Number of terms at each degree of interaction: 1 6 (additive model)
GCV 5.6    RSS 138    GRSq 0.35    RSq 0.65

```

Mars - analysis of iris data (categorical Y)

how does the mars algorithm accommodate for categorical variables?

```

# do i need to add a glm argument into earth?
data(iris)

iris.earth1 <- earth(Species ~ ., data = iris, degree = 1)
summary(iris.earth1, digits = 2, style = 'bf')

iris.earth2 <- earth(Species ~ ., data = iris, degree = 2)
summary(iris.earth2, digits = 2, style = 'bf')

# Rsq continuously increasing with increasing degree
iris.earth3 <- earth(Species ~ ., data = iris, degree = 3)
summary(iris.earth3, digits = 2, style = 'bf')

```

I was unsure how to work with multiclass response variables. I could not find an appropriate link function. I consulted with AI [https://chatgpt.com/s/t_6abe295ac1288191b96da64b81ac0d98]. It advised that it cannot be done efficiently using earth, since earth accommodates for 2 classes at most. It recommended performing an analyses of just 2 species at a time.

General

MARS has several advantages because it can model nonlinear relationships between the predictor and response variables. It can automatically identify important variables and interactions between variables without requiring the user to specify the relationships beforehand. MARS is also relatively easy to interpret compared with some more complex machine learning methods and can be useful for making accurate predictions when the relationships in the data are complex.

However, the model can overfit the data if it becomes too complex, which can result in poor predictions when applied to new data. MARS models can also be more difficult to interpret than simple linear regression models. In addition, MARS can be sensitive to changes in the data and may require additional methods when dealing with multiclass outcomes, such as the three different species in the Iris dataset.

Mini project 2

Summary

This study investigated how the penalty parameter in Multivariate Adaptive Regression Splines (MARS) affects predictive performance as the complexity of the underlying relationship changes. MARS is a flexible non-parametric regression method that uses piecewise-linear hinge functions to model nonlinear effects and interactions. Although this flexibility allows MARS to capture complex relationships, excessive flexibility can lead to unnecessarily large models and overfitting. The penalty parameter in the generalized cross-validation (GCV) criterion controls this trade-off by penalizing model complexity.

A simulation study was conducted using three data-generating relationships representing simple, moderate, and complex levels of complexity. In each simulation, two predictors were generated independently from uniform distributions, with normally distributed errors added to the response. The simple relationship contained a linear effect of one predictor, the moderate relationship added a nonlinear hinge effect, and the complex relationship included two nonlinear effects and an interaction between the predictors. For each complexity level, 100 training data sets of 500 observations were generated, together with independent test data sets of 1,000 observations. MARS models with a maximum interaction degree of two were fitted using penalty values ranging from 1 to 9. Predictive performance was assessed using test-set mean squared error (MSE), while the number of model terms was used as a measure of model complexity.

The results indicated that the relationship between penalization and predictive performance depended on the underlying data-generating structure. For the simple relationship, the lowest mean test MSE was approximately 0.252 at a penalty of 5, with little further improvement at higher penalties. The moderate relationship produced a similar minimum MSE of approximately 0.252 at a penalty of 6, after which performance remained essentially unchanged. In contrast, the complex relationship achieved its lowest mean test MSE of approximately 0.259 at the lowest penalty considered, 1, with test MSE increasing slightly as the penalty increased. Increasing the penalty generally reduced the number of terms selected by the MARS model. This reduction in model size had little effect on predictive performance for the simple and moderate relationships once

their MSE had plateaued, whereas stronger penalization reduced predictive performance for the complex relationship.

Overall, the simulation suggests that the penalty associated with the lowest predictive error can depend on the complexity and structure of the underlying relationship. Simpler relationships benefited from stronger penalization, while the more complex relationship required greater flexibility. However, the differences in mean test MSE were small, and the study was limited to three specific data-generating functions, penalty values from 1 to 9, and a fixed sample size and noise level. Therefore, the results should not be interpreted as establishing a universal optimal penalty for MARS. Further simulations involving a wider range of functions, penalty values, noise levels, and interaction structures would be required to assess the robustness of these findings.

Reflection

I decided on this topic since I was interested in the generalized cross validation statistic, which was a new concept. I realized afterwards that it would have been interesting to contrast these results to that using the standard cross validation criterion.

With regards to the data, simulated data is best for evaluating and comparing numerous conditions since all other aspects can be controlled. The sequence of penalty values was altered numerous times due to the ‘best’ penalty being an upper bound in the simple and moderate cases.

This project solidified my understanding of the generalized cross validation error. Initially, I did not understand the relationship between the penalty and GCV, as well as the relationship between the penalty and the knots and the terms. I understand now that the GCV penalizes both the number of knots and the number of basis functions. When a penalty of 0 is implemented using GCV, it does not imply there is no penalty and regression is not regularized. It implies that the number of basis functions only is penalized; the number of knots is not penalized. This is understood via the complexity formula $C(M) = r + d \times \frac{r-1}{2}$ which is within the GCV calculation. the parameter r represents the number of basis functions. Hence, the second term in the above formula refers to the number of knots (multiplied by the penalty). Another important point I was made to take cognizance of is that the GCV penalty value is only applied as a penalty to the knots. Hence it is known as the penalty per knot. And lastly, GCV caters for and corrects the phenomenon of ever increasing model size resulting in better performance.

Wavelets

Notes

- Wavelets are short, localized oscillating functions used to represent signals and functions at different scales and locations. They can capture both smooth trends and local irregularities.
- A signal is some type of curve. I typically picture a sound wave. If the sound was ‘recorded in a studio’ there would not be any ‘noise’ in the wave. But since there are so many sounds in recordings, the sound is not isolated. We are working with un-isolated sound, and want to predict the isolated sound. We need to develop a function to do this.

- Wavelets have numerous applications such as image compression, noise reduction and signal reconstruction. It is also useful to identify changes in frequency over time. Wavelets are also efficient in feature extraction.
- An objective is to separate a meaningful pattern from noise while preserving important local changes.

Wavelet basis functions:

- The mother wavelet ($\psi(x)$) is the original wave and is used to generate wavelets via translation (shifting along the x axis) and dilation (changing scale/FREQUENCY, ie shrinking or enlarging). $\psi(x)$ is the height of the curve at location x.

The following defines translations and dilations:

$$\psi_{a,b}(x) = \frac{1}{\sqrt{|a|}} \psi\left(\frac{x-b}{a}\right),$$

where a controls the scale (frequency) of the wavelet. Higher values of a result in a wider curve. This is since the height of the curve $\psi(\cdot)$ is going to be reduced at all x (curve will become shorter, and fatter). Smaller values of a will make the peaks look skinnier, hence, narrower. the b parameter translates the curve along the x axis. No scaling occurs when a = 1, and no translation occurs if b = 0.

Derivation of daughter

The daughter wavelet is obtained by transforming the mother wavelet according to

$$\psi_{a,b}(x) = \frac{1}{\sqrt{|a|}} \psi\left(\frac{x-b}{a}\right),$$

where a controls the scale of the wavelet and b controls its location.

Consider the following mother wavelet:

$$\psi(x) = \begin{cases} 1 & 0 \leq x < 1 \\ -1 & 1 \leq x < 2 \\ 0 & \text{otherwise} . \end{cases}$$

To demonstrate how a daughter wavelet is obtained, let $a = 2$ and $b = 3$.

Substituting these values into the transformation equation gives

$$\psi_{2,3}(x) = \frac{1}{\sqrt{|2|}} \psi\left(\frac{x-3}{2}\right).$$

Since $\sqrt{|2|} = \sqrt{2}$,

this becomes

$$\psi_{2,3}(x) = \frac{1}{\sqrt{2}}\psi\left(\frac{x-3}{2}\right).$$

The definition of the mother wavelet must now be applied to the transformed input

$$\frac{x-3}{2}.$$

First part of the mother wavelet

The mother wavelet has value 1 when $0 \leq x < 1$.

For the daughter wavelet, the input to the mother is

$$\frac{x-3}{2}.$$

Therefore, the first part of the daughter occurs when

$$0 \leq \frac{x-3}{2} < 1.$$

Multiplying throughout by 2 gives

$$0 \leq x-3 < 2.$$

Adding 3 gives

$$3 \leq x < 5.$$

Therefore, for $3 \leq x < 5$, the mother wavelet has value 1. Including the normalization factor gives

$$\psi_{2,3}(x) = \frac{1}{\sqrt{2}} \times 1.$$

Second part of the mother wavelet

The mother wavelet has value -1 when $1 \leq x < 2$.

Again, the input to the mother wavelet is

$$\frac{x-3}{2}.$$

Therefore,

$$1 \leq \frac{x-3}{2} < 2.$$

Multiplying throughout by 2 gives

$$2 \leq x-3 < 4.$$

Adding 3 gives $5 \leq x < 7$.

Therefore, for $5 \leq x < 7$, the mother wavelet has value -1 . Including the normalization factor gives

$$\psi_{2,3}(x) = \frac{1}{\sqrt{2}} \times -1.$$

Outside these intervals

The mother wavelet is zero everywhere else. Therefore, the daughter wavelet is also zero everywhere else.

Hence, the resulting daughter wavelet is

$$\psi_{2,3}(x) = \begin{cases} \frac{1}{\sqrt{2}} & 3 \leq x < 5 \\ -\frac{1}{\sqrt{2}} & 5 \leq x < 7 \\ 0 & \text{otherwise} \end{cases}.$$

Thus, the daughter wavelet has been obtained directly from the mother wavelet by applying a scale transformation with $a=2$, a translation with $b=3$, and the normalization factor $1/\sqrt{|a|}$.

The scale parameter $a=2$ makes the daughter wavelet twice as wide as the mother wavelet, while the translation parameter $b=3$ shifts it to the right. The normalization factor changes its amplitude so that the wavelet retains the same height as the mother wavelet.

General

- The *father wavelet* ($\phi(x)$) or scaling function captures the overall structure of a signal. It can be thought of as adding all points together. Important contributor to intercept.
- Orthonormal wavelet bases allow a signal to be represented using wavelet coefficients
- A signal is represented as a combination of scaling functions and wavelet basis functions.
- Wavelet coefficients measure the contribution of individual wavelets to the signal. Detail coefficients are the wavelet coefficients.
- Waves are localized, which means they only act on a portion of the signal.
- Wavelets integrate to 0
- The Discrete Wavelet Transform (DWT) efficiently decomposes a signal into approximation and detail coefficients using a pyramid algorithm.
- Haar wavelets are simple, piecewise-constant wavelets that capture local changes.
- Daubechies wavelets are compactly supported, orthonormal wavelets that can provide sparse representations of functions. Their vanishing moments help represent smooth polynomial behaviour efficiently.

- Contrasting Fourier methods and wavelets:
Fourier tells you which frequencies occur in a signal, but does not tell you where in time they occur.
Wavelets provide both time/location and frequency/scale information, making them useful for signals whose characteristics change over time or space.
- Noise reduction is achieved by shrinking or removing wavelet coefficients that are likely to represent noise rather than meaningful signal variation.
- Hard thresholding sets coefficients below a specified threshold to zero.
- Soft thresholding shrinks coefficients towards zero and sets sufficiently small coefficients to zero.
- The universal threshold is $\lambda = \sigma\sqrt{2\log N}$, where σ is the noise standard deviation and N is the number of observations.
- SURE (Stein's Unbiased Risk Estimate) provides an alternative approach to selecting shrinkage, with soft thresholding related to the LASSO's coefficient-shrinkage principle.
- For an orthonormal wavelet basis matrix W , the transform is $y^* = W^\top y$, and reconstruction is $\hat{f} = W\hat{\theta}$.
- The pyramid algorithm computes the DWT much more efficiently than direct matrix-based computation.
- The wavelet content sounds similar to latent variable models. An example of a typical scenario where latent variable models are used is the cocktail party problem, where the aim is to recover hidden sources from observed mixtures of signals. For example, a microphone may record several people speaking simultaneously, and the individual voices must be separated from the combined recording. In contrast, wavelet methods represent an observed signal as a combination of basis functions at different scales and locations. Although both approaches can be applied to mixed audio signals, their objectives differ - latent variable models are used to identify the underlying sources that generated the mixture, whereas wavelets describe the structure of the observed signal through its localized features at different scales. Therefore, wavelet decomposition does not necessarily separate individual speakers.

I initially found this extremely difficult to conceptualize, especially in the context of regression. However, I now understand that the equivalent of the coefficients in a regular regression expression are the detail coefficients here. And, the predictors are the wavelet basis function. This is regarding wavelet representation of a signal. The coefficients infer how much of that particular wave is contained in the signal.

To assess my understanding, I asked AI to ask me a conceptual question on wavelets.

Question 1

Consider the daughter wavelet

$$\psi_{j,k}(x) = 2^{j/2}\psi(2^jx - k).$$

Explain in words what changing the following parameters does:

- j
- k

My answer:

The parameter j is the dilation parameter, meaning it influences scale (frequency) of the wavelet. Increasing j will result in a wider wave, whereas smaller values will produce a thinner wave. This is because increasing j will increase 2^j , resulting in an increased height compared to the original function, making the wave appear shorter and wider. The opposite is true for low values of j .

The location parameter is k . This will control where on the original domain this function will act. For positive values of k , the wavelet will shift right, and for negative k , the wave will move to the left

Reflection:

AI flagged this as incorrect, stating that my understanding was correct except my direction was reversed. The reason is that j is not directly the scale parameter, instead it merely controls the scale parameter. It states that a, the scale parameter in this case is actually 2^{-j} . This explains why the direction is reversed. The location parameter is actually $k2^{-j}$, meaning that increasing (k) increases the location parameter and therefore shifts the wavelet to the right.

Practice

Question 1

A sinusoidal signal was created using 16 equally spaced points between 0 and 2π . The Haar father wavelet with two points was scaled to have a norm of 1. The Haar wavelet was then convolved with the signal to demonstrate the effect of wavelet convolution. The original signal and the resulting convolution were plotted to compare their behaviour.

Difference Between the Two Plots

The first plot shows the original sinusoidal signal, where the x axis represents the original locations between 0 and 2π , and the y axis contains the corresponding values of $\sin(x)$.

The second plot shows the result of convolving the signal with the Haar father wavelet. The convolution combines neighbouring values of the original signal using the Haar wavelet weights. Since the two-point Haar wavelet has equal positive weights, the convolution effectively produces a local weighted sum of neighbouring signal values. The convolution also has 17 values because `type = "open"` includes the boundary values.

```
x <- seq(0, 2*pi, length = 16)
y <- sin(x)
par(mfrow = c(1, 2))
plot(x, y, type = 'l', main = "original")
```

```
#haar <- c(1,1, rep(0, 14))

# father
haar <- c(1/sqrt(2),1/sqrt(2))

# includes boundary
convolution <- convolve(x = y, y = haar, conj = T, type = 'open')

plot(1:length(convolution),convolution, type = 'l', xlab = "x", main =
"convolved")
```

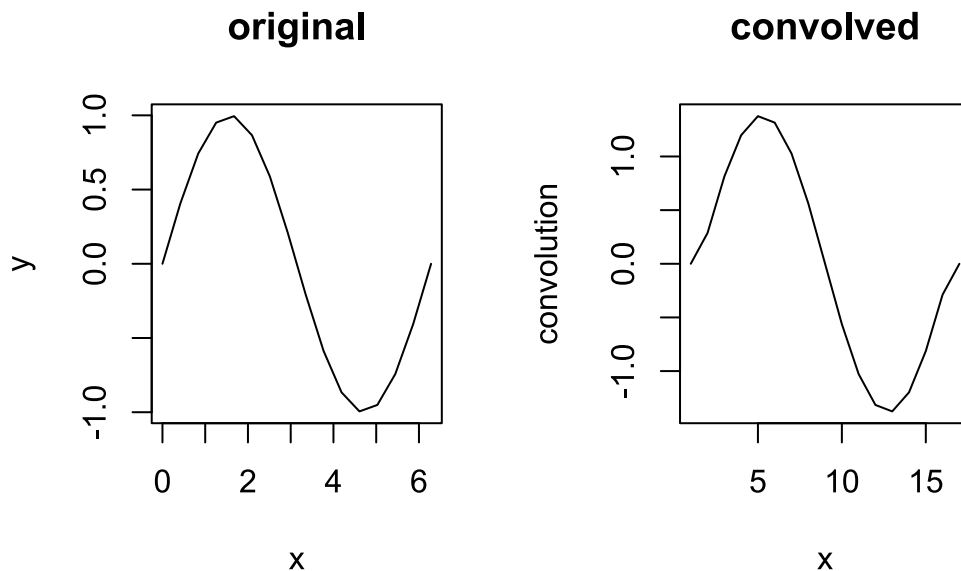


Figure 3: Plots of the original and convolved signal.

After having difficulty with the convolve function, I investigated the different methods that this function uses. A vector of (1, 2, 3, 4, 5) was used as input. The first method was the open method, which produced an output of (1, 3, 5, 7, 9, 5). This is consistent with keeping the bounds, and adding every 2 consecutive numbers together. For example, 1 is the bound, $3 = 1+2$.

The circular method was tested next with a father basis of (1,1,1,1,1). Its output was (55, 45, 40, 40, 45). This method shifts and 'wraps around' the values. 55 was calculated as $(1)(1) + (2)(2) + (3)(3) + (4)(4) + (5)(5)$. The next value was calculated after a shift and wrap: $(5)(1) + (1)(2) + (2)(3) + (3)(4) + (4)(5) = 45$.

```
# investigating the different methods of convolving
```

```
x <- c(1:5)
father1 <- c(1,1)
father2 <- c(1:5)

open_c <- convolve(x, father1, type = 'open')

circular_c <- convolve(x, father2, type = 'circular')

filter_c <- convolve(x, father1, type = 'filter')
```

Question 2

```
library(wavethresh)
```

Loading required package: MASS

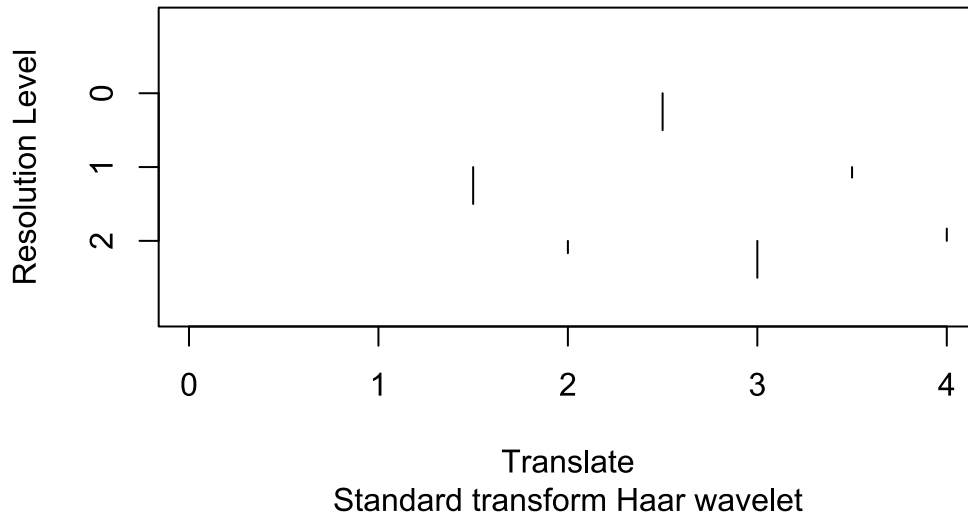
WaveThresh: R wavelet software, release 4.7.2, installed

Copyright Guy Nason and others 1993-2022

Note: nlevels has been renamed to nlevelsWT

```
y <- c(1, 1, 7, 9, 2, 8, 8, 6)
wd_y <- wd(y, filter.number = 1, family = "DaubExPhase")
plot(wd_y , scaling = "by.level")
```

Wavelet Decomposition Coefficients



```
[1] 4.242641 7.000000 2.121320
```

```
D0 <- accessD(wd_y, level = 0)
D1 <- accessD(wd_y, level = 1)
D2 <- accessD(wd_y, level = 2)
```

Question 3

The Mexican Hat mother wavelet is given by

$$\psi(t) = (1 - t^2) \exp\left(-\frac{t^2}{2}\right).$$

Daughter wavelets are obtained by applying the general translation and dilation formula

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \psi\left(\frac{t-b}{a}\right),$$

where a is the scale parameter and b is the translation parameter.

Substituting the Mexican Hat mother wavelet into the daughter wavelet formula gives

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \left[1 - \left(\frac{t-b}{a}\right)^2 \right] \exp\left[-\frac{1}{2} \left(\frac{t-b}{a}\right)^2\right].$$

Therefore, the daughter wavelet of the Mexican Hat function is

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \left[1 - \left(\frac{t-b}{a} \right)^2 \right] \exp \left[-\frac{(t-b)^2}{2a^2} \right]$$

The parameter a controls the scale or width of the wavelet. Increasing a produces a wider wavelet, while decreasing a produces a narrower wavelet. The parameter b controls the location of the wavelet, with increasing b shifting the wavelet to the right.

I have come to understand that in order to derive the daughter wavelet or to establish how the wave would change when parameters are altered, the wave function would first have to be put into the general formula where the scale and location parameters are expressed explicitly as a and b .

Gaussian processes

Notes

A Gaussian Process (GP) is a probability distribution for a function. Instead of specifying a fixed functional form, such as a straight line, a Gaussian Process defines a probability distribution over possible functions. A GP produces realizations of a function. The univariate equivalent of realization is observation.

The Gaussian Process regression model is

$$y(x) = f(x) + \epsilon,$$

where

$$\epsilon \sim N(0, \sigma_n^2)$$

represents observation noise.

The underlying true function is assumed to follow a Gaussian Process:

$$f(x) \sim GP(m(x), k(x, x')),$$

where:

- $m(x)$ is the mean function.
- $k(x, x')$ is the covariance function, also called the kernel.
- The kernel determines how strongly function values at different input values are correlated.

For a set of observations, this gives

$$\mathbf{y} \sim MVN(\mathbf{m}, K + \sigma_n^2 I).$$

The Covariance Function (Kernel)

The covariance function describes the covariance between the function values at two input locations x and x' .

A common choice is the squared exponential kernel:

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right).$$

A larger ℓ means that observations remain correlated over a greater distance, producing a smoother function.

- σ_f controls the variability of the function values around the mean, ie. the amplitude
- ℓ controls the length scale. It controls how quickly the correlation between two function values decreases as the distance between x and x' increases. Large values of this parameter implies a smoother function. The figure below demonstrates this phenomenon.

Thus, σ_f mainly controls the vertical scale of variation, while ℓ controls the smoothness of the function.

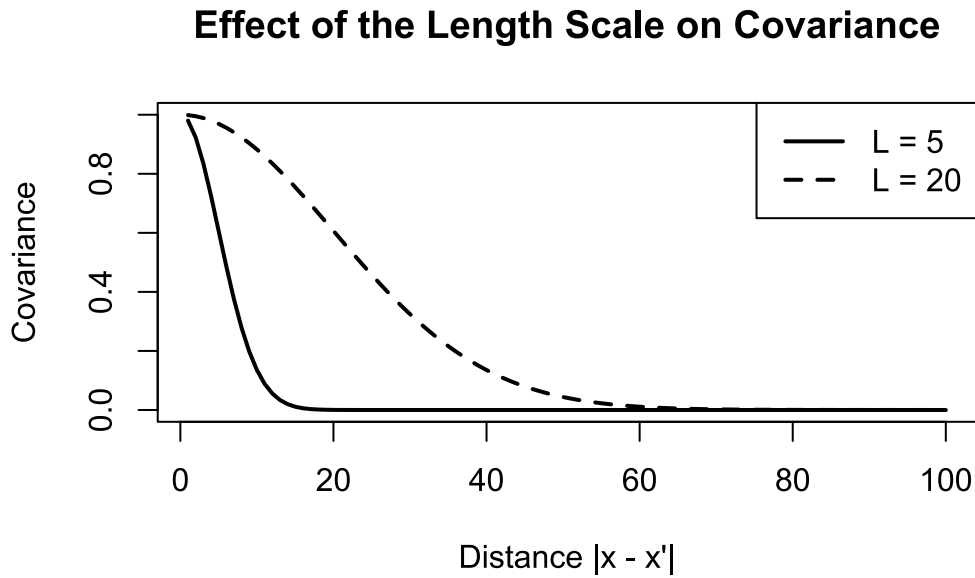


Figure 4: comparison of length scale effect in kernel function

Comparison with Ordinary Regression

Ordinary linear regression assumes

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i,$$

where

$$\epsilon_i \sim N(0, \sigma^2).$$

Therefore,

$$\mathbf{y} \sim MVN(\boldsymbol{\mu}, \sigma^2 \mathbf{I}).$$

The key difference is that ordinary regression specifies the mean function using parameters such as β_0 and β_1 . In contrast, a Gaussian Process specifies relationships between observations through a covariance function.

A Gaussian Process as a Random Function

- A useful interpretation is that a Gaussian Process is a probability distribution over possible functions.

Suppose we have input values $x_1, x_2, \dots, x_n$.

The corresponding function values

$$f(x_1), f(x_2), \dots, f(x_n)$$

have a multivariate normal distribution:

$$\mathbf{f} \sim MVN(\mathbf{m}, K).$$

The covariance matrix K is constructed using the kernel:

$$K_{ij} = k(x_i, x_j).$$

- Therefore, the kernel determines which function values are strongly related and which are weakly related.
- Consequently, the kernel determines which possible functions are considered plausible under the Gaussian Process.
- An important aspect for understanding is recognizing that the mean is a function, and the covariance structure is also a function (kernel). In conventional regression, the mean and covariance is a vector and matrix respectively.

Kriging

- Kriging uses a Gaussian process to model spatial data.

I did not quite understand when this would be used, or why it is beneficial. I have consulted with AI [https://chatgpt.com/s/t_6abe92af80f881918097098644d2b04e]. The main idea is that it is used when there is spatial data. Usually, there is spatial autocorrelation in spatial data since nearby points tend to have similar characteristics. The kernel function can efficiently establish the covariance between location. Furthermore, predictions can be made.

I now understand that spatial data is perfectly applicable for a gaussian process since there is a spatial domain (implying it is functional data), and the covariance between observations changes over the landscape.

Applications of Gaussian Processes

Gaussian Processes can be used in several settings:

- Autocorrelated errors in time-series, spatial data and other longitudinal models.
- Can predict values for data not observed.

- Optimisation where the GP can approximate an unknown function and help identify promising values of the inputs.
- One important advantage of Gaussian Processes is that they provide uncertainty estimates for predictions. Rather than producing only a point prediction, they produce a posterior distribution. Therefore, Gaussian Process regression provides both a prediction and a measure of uncertainty about that prediction.
- There are two main approaches to estimating Gaussian Process models - Bayesian and marginal likelihood. The Bayesian approach treats unknown parameters as random variables and combines prior information with the likelihood to obtain posterior distributions. Alternatively, the parameters of the Gaussian Process can be estimated by maximizing the marginal likelihood. Parameters to estimate include length scale, observation noise and function noise.
- A major limitation of Gaussian Processes is their computational cost. For n observations, the computational complexity is typically approximately $O(n^3)$. This arises primarily from operations involving the $n \times n$ covariance matrix, such as matrix inversion. Therefore, Gaussian processes can become computationally expensive when the number of observations is large.

The central idea behind Gaussian Processes can be summarized as

Kernel $\rightarrow$ Covariance $\rightarrow$ Relationship between function values $\rightarrow$ Distribution over possible functions

Practice

Question 1

```
library(mvtnorm)
set.seed(1)
x <- seq(0, 10, length = 100)
d = abs(outer(x, x, "-"))

GP <- function(nreps, mu, sigma){

  mx <- mu

  Sigma_SE = sigma # squared exponential kernel

  set.seed(1)

  y_mat <- matrix(0, nrow = length(x), ncol = nreps+1)
  for (i in 1:nreps){
    y_mat[, i] <- mvtnorm::rmvnorm(1, mean = mx, sigma = Sigma_SE)
  }

  plot(x, y_mat[,1], type = 'l', col = 'purple', lwd = 2, ylim = c(min(y_mat),
max(y_mat)), main = 'Gaussian process', ylab = "Y", xlab = "x")
}
```

```

for (i in 2:nreps){
  lines(x,y_mat[,i], col = i, lwd = 2)
}

y_mat[, nreps+1] <- mvtnorm::rmvnorm(1, mean = mx, sigma = Sigma_SE)

vals <- y_mat[seq(0, 100, by = 5), nreps+1]
x_vals <- seq(0, 10, length = 20)

points(x_vals, vals, pch = 19, col = 1)

set.seed(8)
eps = rnorm(20, 0, 1)

observations <- vals+eps

points(x_vals, observations, pch = 20, col = 'grey20')

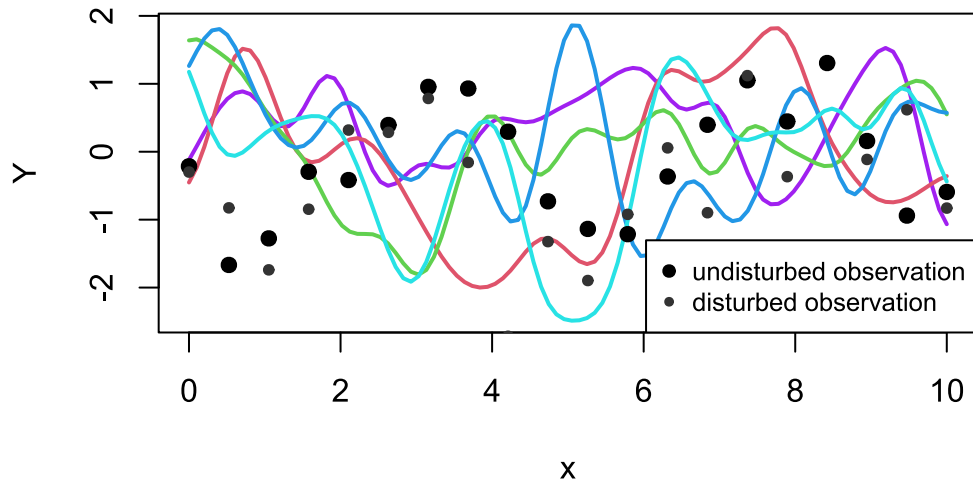
legend('bottomright', pch = c(19, 20), col = c('black', 'grey20'), legend =
c("undisturbed observation", "disturbed observation"), cex = 0.80)

}

GP(nreps = 5, mu = rep(0, times = 100), sigma = exp(-d^2 / (2 * 0.5^2)))

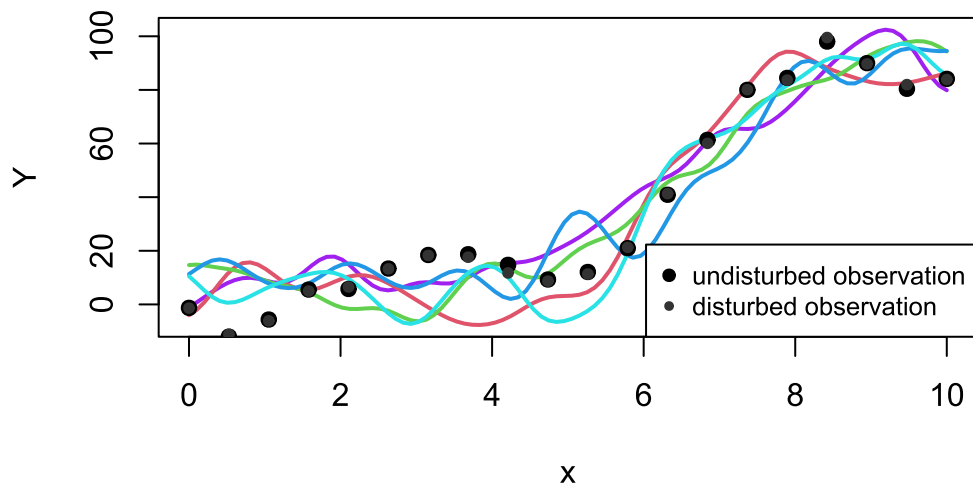
```

Gaussian process



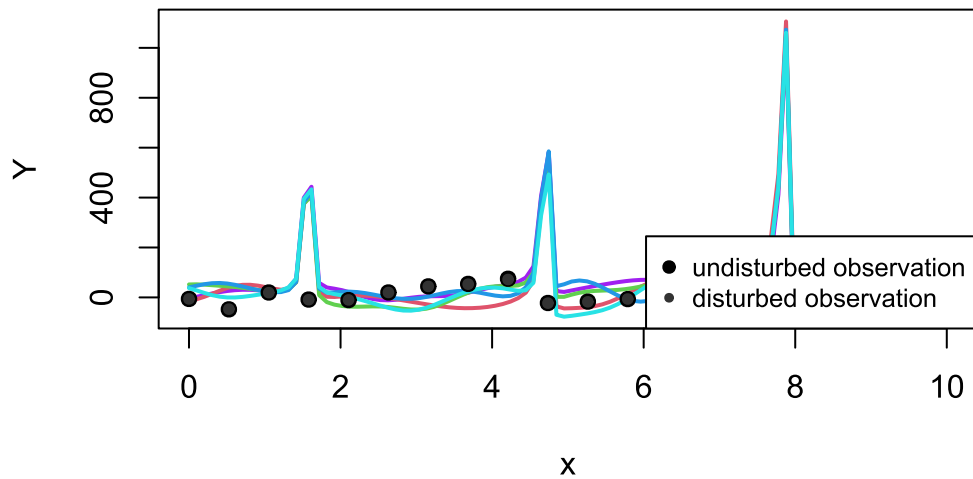
```
GP(nreps = 5, mu = (sin(x)+x)^2, sigma = 80*exp(-d^2 / (2 * 0.5^2)))
```

Gaussian process



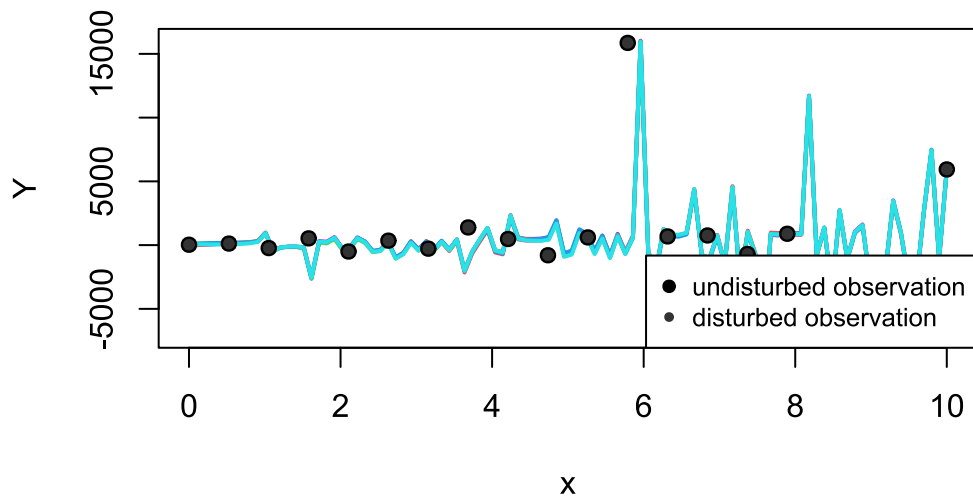
```
GP(nreps = 5, mu = (tan(x)+x)^2, sigma = 1000*exp(-d^2 / (2 * 0.5^2)))
```

Gaussian process



```
GP(nreps = 5, mu = x*100/sin(2+x^3), sigma = 5000*exp(-d^2 / (2 * 0.5^2)))
```

Gaussian process



Question 2

Fit a GP using maximum likelihood and simulate from the posterior.

```

library(mvtnorm)

# dat from on amathuba page
tt <- 1:6
xt <- c(0.9, 3.8, 5.2, 6.1, 7.5, 9.6)
yt <- c(0.1, 1.2, 2.1, 1.1, 1.5, 1.2)

# kernel function (pg 2)
se_kernel <- function(x1, x2, sigma_f, ell) {
  sigma_f^2 * exp(-(outer(x1, x2, "-")^2) / (2 * ell^2))
}

# Neg log marginal likelihood ((follow formula in course notes, pg))

log_marginal_likelihood <- function(theta, x, y) {

  # values being estimayed
  sigma_f <- exp(theta[1])
  ell <- exp(theta[2])
  sigma_n <- exp(theta[3])

  n <- length(x)

  #####

  ## can investigate different kernel functions****

  # calc the covar mat for the fns (pg3)
  K <- se_kernel(x, x, sigma_f, ell)

  # pg 3, add noise to covar mat
  Ky <- K + sigma_n^2 * diag(n)

  Ky_inv <- solve(Ky)

  # second term in log like (pg5)
  log_det <- determinant(Ky, logarithm = TRUE)$modulus

  logLik <- -0.5 * t(y) %*% Ky_inv %*% y -
    0.5 * log_det -
    0.5 * n * log(2 * pi)

  return(-as.numeric(logLik))
}

```

```

# actually need to optimize the loglike
fit <- optim(par = log(c(1, 2, 0.5)), fn = log_marginal_likelihood, x = xt, y =
yt)

# Est paras

theta_hat <- exp(fit$par)

sigma_f_hat <- theta_hat[1]
ell_hat <- theta_hat[2]
sigma_n_hat <- theta_hat[3]

#sigma_f_hat
#ell_hat
#sigma_n_hat
# use these values to get estimates for covar mat and mean

# fitted covar mat with estimated params
K <- se_kernel(xt, xt, sigma_f_hat, ell_hat)

# need to account for the noise
Ky <- K + sigma_n_hat^2 * diag(length(xt))

Ky_inv <- solve(Ky)

## new x vals
x_new <- seq(min(xt), max(xt), length.out = 100)

# covar mat for prediction--
# pg 4 - covar with existing x and new x input
K_star <- se_kernel(x_new, xt, sigma_f_hat, ell_hat)

# pg 3 - covar with existing x and new x input
K_starstar <- se_kernel(x_new, x_new, sigma_f_hat, ell_hat)

# Posterior mean (pg4)
posterior_mean <- K_star %*% Ky_inv %*% yt

```

```
# 9. Posterior covariance pg(4)
posterior_covariance <- K_starstar - K_star %*% Ky_inv %*% t(K_star)

## now use these to simulate dat
set.seed(8)

posterior_samples <- mvtnorm::rmvnorm(
  n = 10,
  mean = as.vector(posterior_mean),
  sigma = posterior_covariance
)
```

The Figure below shows the posterior predicted mean, the posterior samples and the observed data. There were fairly few data points to work with. However, the predicted mean function appears to be extremely well estimated in relation to the posterior curve samples.

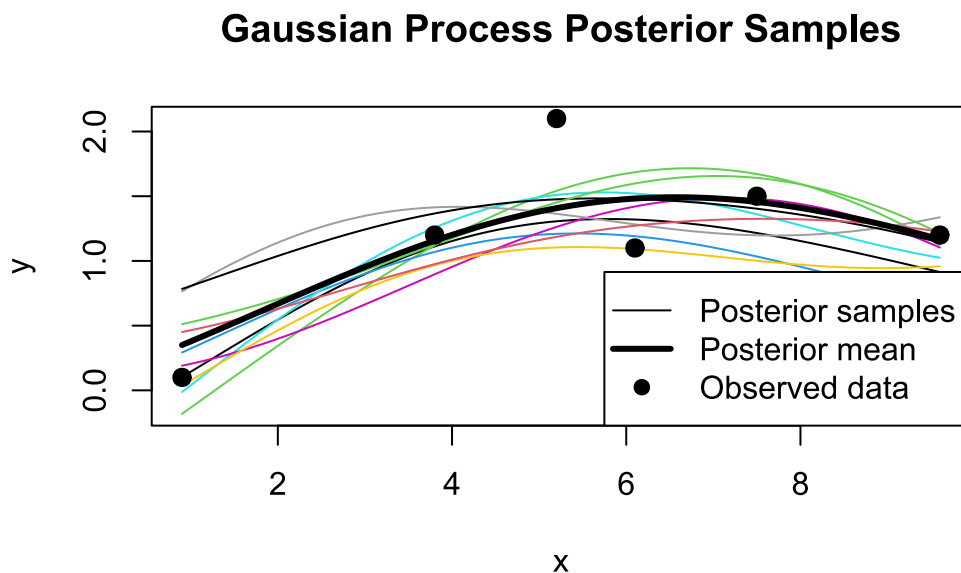


Figure 5: Gaussian process fit using maximum likelihood.

This question took a significant amount of time and patience. I found the course notes absolutely crucial for the formulas to implement. The procedure was confusing, and I had trouble identifying and setting up a workflow. I consulted with AI [https://chatgpt.com/s/t_6abeaa2fdeac8191a4e9032a4c7836c6], to gain an understanding of the problem and the steps required. This also helped immensely to understand the order in which to do things since the course notes do not present the formulas in the order they should be implemented. However, the code was implemented from formulas already provided in the notes. I do feel like I would need a lot more practice of problems from different perspectives to gain a good global and inherent understanding of the scenario.

Regression

Notes

- For the sake of efficient pdf rendering, the inner product is instead represented using roundd brackets rather than the conventional angle brackets.
- General linear algebra principles were revisited. This was extremely beneficial to me for my understanding of future concepts in the course, as well as in the methods that had already been covered in section 1 of the course.
- Derivation of projection matrix for projecting onto a line Consider x , y and the projection of y onto x , z . We can express z as a multiple of x : $z = cx$. Furthermore, the difference between the y and z vectors can be calculated as $(y-z)$. Since we are considering an orthogonal projection of y onto x , the inner product of x and $(y-z)$ is 0 (perpendicular). The following shows the process of reaching the projection matrix:

$$\langle (y - z), x \rangle = 0$$

$$\langle y, x \rangle - \langle z, x \rangle = 0$$

$$\langle y, x \rangle = \langle z, x \rangle$$

Substitute $z = cx$:

$$\langle y, x \rangle = \langle cx, x \rangle$$

$$c \langle x, x \rangle = \langle x, y \rangle$$

This implies that

$$c = \frac{\langle x, y \rangle}{\langle x, x \rangle}$$

$$\frac{x^T y}{(x, x)}$$

But $z = xc$. This implies that

$$z = x \times \frac{x^T y}{(x, x)}$$

$$\frac{xx^T}{(x, x)} \times y$$

The projection matrix to project y onto x orthogonally in this scenario is $\frac{xx^T}{(x, x)}$.

- Projection matrix of y onto subspace x was also explored. This is useful since most real world scenarios are multi-dimensional.

Consider a matrix X of dimensions $n \times p$. Projection of Y onto x subspace must be a linear combination. once again, let z represent the projection. Therefore, $z = Xc$.

We start with

$$z = Xc.$$

Due to orthogonality,

$$\langle (y - z), X \rangle = 0$$

$$\langle y, X \rangle - \langle z, X \rangle = 0$$

Considering $z = Xc$,

$$\langle y, X \rangle = \langle Xc, X \rangle$$

$$\langle X, y \rangle = c \langle X, X \rangle$$

$$X^T y = c X^T X$$

$$c = \frac{X^T y}{(X, X)}$$

But $z = Xc$, hence

$$z = \frac{XX^T}{(X, X)} \times y$$

$$z = \frac{XX^T}{X^T X} \times y$$

$$z = X(X^T X)^{-1} X^T y$$

- Linear regression concepts were recapped.
- The standard linear regression model is $y = \beta_0 + \sum_{j=1}^p x_{ij}\beta_j + \epsilon_i$. A slightly different model is $y = \beta_0 + \sum_{j=1}^p x_{ij}^2\beta_j + \epsilon_i$ is still linear because *coefficients* are raised to power 1, not the variables.
- The Gram schmidt procedure is used to find orthogonal bases for non-orthogonal variables. I understood this section and regression by successive orthogonalization from the lectures. The equations also made the concept more clear.
- In the case of multicollinearity, when predictors are highly correlated, the angle θ between them becomes very small, making the denominator close to zero. This amplifies small changes in the data, causing the coefficient estimate to become highly unstable and have a large variance. If $\theta=0$, the predictors are identical, so they provide no separate information and their individual coefficients cannot be uniquely estimated.

- MSE consists of reducible and irreducible error. The irreducible error is made up of the variability in the data. The reducible error consists of variance and a squared bias measure of the estimated statistics.
- Bias measures how far an estimator's average prediction is from the true value, while variance measures how much it changes across different samples. Bias and variance are indirectly proportional. A more flexible model typically has lower bias but higher variance. A simpler model typically has higher bias but lower variance.
- Subset selection is important for selecting only a few predictors to include in a model, instead of having an unnecessarily full model. Subset methods such as forward and backward stepwise explicitly drop/exclude certain predictors from the model. Shrinkage methods such as ridge regression lower the contribution of unnecessary variables.
- Standard errors of beta coefficients are important since the coefficient values calculated are only valid for the particular data set at hand. Another data set of the same variables, number of observations etc can and will very likely produce at least slightly different coefficient values. This is just due to sampling variability, and as a result of this, there is a slightly different model fit to each data set.
- in ridge regression, the coefficients are calculated as:

$$\hat{\beta}_{\text{ridge}} = \operatorname{argmin}_{\beta} \left\{ \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}.$$

In matrix notation:

$$= \operatorname{argmin}_{\beta} \{ (Y - X\beta^T)(Y - X\beta) + \lambda \beta^T \beta \}.$$

These are in the lagrangian form. the penalty parameter λ controls how severely the coefficients are penalized.

Overall

Mini project 3

Summary

This mini study investigated the predictive performance of five regression approaches - forward stepwise selection, backward stepwise selection, Ridge regression, LASSO, and Pareto Optimization Subset Selection (POSS)— under different simulated regression conditions. The main objective was to examine how sample size and correlation between predictors influence the performance of these methods and to assess whether POSS provides a useful alternative to conventional variable-selection and penalized regression approaches.

Data were simulated using ten predictor variables, of which only three had non-zero true coefficients. The response was generated from a linear regression model with normally distributed errors. Three simulation scenarios were considered: varying sample size ($n = 50, 100, 250, 500$),

varying predictor correlation ($\rho = 0, 0.3, 0.7, 0.9$), and combinations of sample size and correlation. Each data set was divided into training and test sets using an 80:20 split. Model performance was evaluated using test-set Root Mean Squared Error (RMSE) and R^2 . POSS was implemented as a stochastic multi-objective optimization procedure that simultaneously maximized training R^2 and minimized the number of predictors, with a maximum subset size of five predictors.

The results showed that no single method consistently produced the same relative performance across all simulated conditions. When sample size varied, POSS performed competitively for $n = 100$ and $n = 250$, while LASSO was consistently competitive. At $n = 50$, Ridge produced the lowest RMSE in the particular simulated data set, whereas differences between most methods became relatively small at $n = 500$. Predictor correlation also influenced performance. The methods were relatively similar when predictors were independent, while larger differences occurred under stronger correlation. In the simulated correlated data sets, LASSO was generally competitive, whereas Ridge produced relatively higher RMSE and lower test-set R^2 . The combined sample-size and correlation scenarios produced more variable results. This demonstrated that performance depended on the specific conditions that generated the data.

The POSS Pareto-front analysis provided an additional perspective by illustrating the trade-off between model complexity and training R^2 . The resulting fronts were very similar across sample sizes, although some variation occurred for subsets containing two and three predictors. Importantly, the Pareto fronts represented the non-dominated solutions identified within 1,000 stochastic optimization iterations and therefore cannot be regarded as exhaustive global Pareto fronts.

Overall, the study demonstrates that regression-method performance can vary according to sample size, predictor correlation, and the particular simulated sample. The main limitation was that each simulation scenario was evaluated only once. This implies that the observed differences may partly reflect random variation. Repeated simulations, alternative data structures, and investigation of different POSS optimization settings would provide a stronger basis for assessing the general performance of the methods.

Reflection

I decided on this topic after encountering the paper by Qian et al. (2015). I am naturally very interested in the field of optimization. Furthermore, I have encountered Pareto optimization in another UCT course this year. Hence, this paper caught my attention for applying it in a completely different context compared to what I was used to. I found the concept of POSS to be understandable.

During the implementation I faced a few difficulties. The main issue was that I was not able to perform the entire analysis iteratively. By this I mean performing each of the methods numerous times so that a distribution of performance values could be collected. However, when I attempted to perform this iteratively, R always malfunctioned, and the process was never able to complete. This is the cause of the main limitation in my study. I am not able to advise on the average or variation of the statistics collected. Hence, it is unknown if the results are general or subject to random variability.

The choices for the number of variables and true non-zero coefficients were arbitrary. For the varying sample sizes and correlations, I attempted to have values that would allow me to investi-

gate both extremes of the spectrum. Although the maximum number of observations could have been infinitely large since the data is generated, I constrained the maximum sample size to a number of observations that many real world data sets are (n=500).

I would really like to investigate this pareto optimization method using different numbers of objective functions, different objective functions and various combinations thereof.

Functional Data analysis

Notes

- Functional data analysis deals with data in which an observation or response or both, is a function rather than a single scalar value. Conventional regression concerns scalar responses regressed on scalar predictors.
- Non-parametric Scalar on Function (SoF) regression is in syllabus. Initial examples of applications were really useful in understanding the concept of scalar on function regression.
- Conventional (SoS) regression: $\mathbf{Y} = \mathbf{X}\beta + \epsilon$.
- SoF regression: $\mathbf{Y} = \mathbf{r}(\mathbf{X}_i) + \epsilon_i$, where $\mathbf{r}(\mathbf{X}_i) = \mathbf{E}[\mathbf{Y} \mid \mathbf{X} = \mathbf{x}]$ is known as the regression operator.
- Morris (2015) defines the equation as $Y_i = \beta_0 + \int_i X_i(t)\beta(t)dt + \epsilon_i$ for all i. He also describes that this can easily be manipulated to accommodate non-continuous responses (such as binary data) by wrapping the response in a link function (such as a logit function).
- Assumptions of model include that the conditional mean of ϵ_i is zero for every X_i ($E[\epsilon_i \mid X_i] = 0$). This is different from independence because zero conditional mean only requires the errors to average to zero for each value of X_i , while independence requires X_i and ϵ_i to have no relationship at all. The second assumption is that the regression operator fulfills continuity and Lipschitz conditions.
- The continuity requirement is that the response must be continuous.
- The Lipschitz of order k condition is that a change in input of $x_1 - x_2$ (Where these are 2 points from a function f) cannot yield an absolute change in the function f of more than a factor of k. For example:
Consider $f(x) = 6x$. Consider points x_1 and x_2 from the predictor variable of function f. The following shows that f(x) is Lipschitz of order 6:

$$|f(x_1) - f(x_2)| = |6x_1 - 6x_2| = 6|x_1 - x_2|.$$

This implies that a change in the input will result in the response changing by at most a factor of 6.

To demonstrate this, consider the scenario where $x_1 = 12$ and $x_2 = 5$. Therefore

$$f(x_1 = 12) = 72 \text{ and } f(x_2 = 5) = 30.$$

The difference in the input values is $x_1 - x_2 = 12 - 5 = 7$.

The difference in the output values is $f(x_1) - f(x_2) = 72 - 30 = 42$ which is equal to $k \times (x_1 - x_2)$ where $k = 6$.

- the Lipschitz factor of k influences the smoothness of the curve. This is since it can act as a restrictor in how drastically the response changes from between inputs. Smaller values of k imply smoother curves since they can only change by a smaller factor.
- The Nadarya-Watson equation is originally $r(x) = \frac{\sum_{i=1}^n y_i K_h(x-x_i)}{\sum_{i=1}^n K_h(x-x_i)}$ and dealt with scalar values for y and x variables. An updated version now accommodates for functional data:

$$\hat{r}(X) = \frac{\sum_{i=1}^n y_i K\left(\frac{d(X, X_i)}{h}\right)}{\sum_{i=1}^n K\left(\frac{d(X, X_i)}{h}\right)}$$

where the response values (y_i) are still scalar, but now X and X_i are functions. $K()$ is a kernel function. $d()$ is a distance function. h is the bandwidth parameter, and influences smoothness. High bandwidth values will result in a smoother overall function since it will accommodate more observations (curves/spectra) as neighbours and will ‘average’ over all of these. Lower h values cause the distance to inflate, resulting in a lot of the other observations being disregarded and not being used in the averaging. The denominator is simply a normalizing factor.

- Bias spread composition: $r(x) - \hat{r}(x) = r(x) - E[\hat{r}(x)] + E[\hat{r}(x)] - \hat{r}(x)$.

Bias: $r(x) - E[\hat{r}(x)]$

Bias depends on regularity of r . I had difficulty understanding the relationship between the Lipschitz condition and bias. After consulting with AI [https://chatgpt.com/s/t_6abd683b1ee48191b4e0ca91b2394f4b], I understand the following:

Lipschitz infers how quickly $r(x)$ changes.

h infers how far away from X we should look when estimating $r(x)$.

If $r(x)$ changes slowly and we only look nearby (h small), then the estimate is close to the true form and bias will be low.

If $r(x)$ changes quickly or we consider distant observations (larger h), the estimate can be further from the truth, resulting in higher bias.

Spread: $E[\hat{r}(x)] - \hat{r}(x)$. As dimension/complexity increases, spread is higher. I did not quite understand how higher dimensionality leads to greater spread. AI helped me conceptualize this [https://chatgpt.com/s/t_6abd6944693c8191b30b1fec4b118d2c]. I found the example the most useful: if there is only 1 dimension (predictor) of height, it is fairly easy to find other people who have a similar height to you. However, when considering 5 different variables (height, weight, age, income, gender), it is considerably more difficult (rare) to find people that have similar values to you on *all* the variables. This method is unfortunately subject to the curse of dimensionality.

- It is also difficult to choose the appropriate bandwidth. Hyperparameter tuning can be considered for this.
- R package used: `fda`

- This method is advantageous since it can incorporate outliers instead of having to remove them. Outliers are simply downweighted. It also allows for mass data processing - conventional regression (SoS) would reduce each observation down to their mean or median (or some other summary statistic), which does not allow the pattern to be observed.
- After exploring a few FDA research papers, I have realised there are so many variations of functional data analysis. Apart from SoS (scalar on scalar) and SoF (scalar on function), the other main branches are FoS (Function on scalar) and FoF (function on function). For each of these, there is further manipulation and computation required for a combination of types. For example, in the FoF scenario, there may be a second predictor which is scalar, or even a second functional predictor. This requires more care than a simple FoF regression.
- **Question: What is longitudinal data analysis and how is it different from functional data analysis?**

Longitudinal data is data where numerous response measurements have been recorded for each observation (Fitzmaurice et al., 2008). Diggle and Taylor-Robinson (2024) mention that longitudinal data in the epidemiology department typically consist of measurements over time. Furthermore, these time-based measurements can be of the predictor variables or the outcome variables.

This does not sound any different from the various kinds of functional data. Since I could not find any definitive differentiation between longitudinal and functional data, I am inclined to believe that they include the same data structures, but are possibly collected in different contexts/scenarios or study disciplines. The analysis mentioned by Fitzmaurice et al. (2008) also resemble the methods of functional data analysis, such as function on scalar regression. I clarified with AI [https://chatgpt.com/s/t_6abd7ef3ccb08191880a16036c996dec]. The one aspect that is different is that LDA considers the data discrete, whereas FDA considers the data to be of functional form.

Mini project 4

Summary

This mini project explored function-on-scalar regression, a form of functional data analysis in which the response is observed as a function over time while the predictors are scalar variables. Unlike conventional regression, where each observation has a single response value, functional regression allows each individual to have multiple response measurements. This type of modelling is useful in applications where measurements are collected repeatedly over a continuous domain, such as time.

The project first considered the mathematical formulation of function-on-scalar regression. For an individual observation i , the functional response was modelled as

$$Y_i(t) = \beta_0(t) + \beta_1(t)X_i + \epsilon_i(t),$$

where X_i is a scalar predictor, $\beta_0(t)$ is a time-varying intercept function, $\beta_1(t)$ is a time-varying slope function, and $\epsilon_i(t)$ represents random error. It was shown that, at each measurement point

t_k , the model can be estimated using ordinary least squares. Thus, the coefficient functions can initially be estimated by fitting a separate linear regression at every value of t .

To demonstrate this approach, a simulated data set containing 100 observations and 100 response measurements per observation was generated. The predictor was drawn from a standard normal distribution, while the response was generated using known coefficient functions,

$$\beta_0(t) = 2 + \sin(2\pi t) \quad \text{and} \quad \beta_1(t) = 2t,$$

with normally distributed random error having standard deviation 0.5. A separate linear regression was then fitted at each of the 100 time points, producing estimated intercept and slope functions. These estimates were subsequently used to obtain predicted response curves for each observation.

The results showed that the point-wise regression approach recovered the underlying coefficient functions reasonably well. The RMSEs between the true and estimated coefficient functions were 0.044 for the intercept and 0.055 for the slope, indicating relatively accurate estimation. The RMSE between the observed and predicted response curves was 0.495, which was close to the specified error standard deviation of 0.5. This was expected because the fitted response represents an estimate of the conditional mean and therefore does not reproduce the observation-specific random error contained in the simulated responses.

An important limitation of the point-wise approach was also identified. Since each coefficient was estimated independently at each measurement point, neighbouring coefficient estimates were not constrained to be similar. This resulted in some local fluctuations, or wiggleness, in the estimated coefficient functions. Although the underlying functions were smooth, sampling variability at individual time points could therefore produce irregular estimates.

The project consequently considered an extension using basis functions and regularization. Instead of estimating each coefficient independently, the coefficient functions can be represented as combinations of basis functions:

$$\beta_j(t) = \sum_{l=1}^K b_{jl} \phi_l(t),$$

where $\phi_l(t)$ represents a basis function and b_{jl} is its corresponding coefficient. A roughness penalty based on the integrated squared second derivative can then be added to the objective function to discourage excessively irregular coefficient functions.

Reflection

Once exposed to the various other methods of functional data analysis during my reading, I became interested in how the other algorithms work and where they are applied. I decided that since Scalar on Function regression was covered in the module, exploring the ‘opposite’ would be a natural extension of the course material.

I did find the method difficult to implement. I also did not take cognizance of the fact that the one X (predictor) value for observation i is used to calculate *all* the response values for observation i .

Regarding my choices for the coefficient functions, I wanted to investigate moderately complex functions. Hence, I added a sine function, hoping for cyclical behaviour. I later realised that the undulating pattern still remained without the sine function. And I just included a standard linear relationship for the other to keep it simple. I also thought a linear and non-linear combination of functions would be interesting to estimate and compare their performances. They were both estimated quite well. I would have liked to apply the improved method described at the end of the study. However, time did not permit. Overall, I think that Functional data analysis is an extremely useful field, especially since we are in an extremely data-rich era.

References

Regression

- - Qian, C., Yu, Y. and Zhou, Z.H., 2015. Subset selection by Pareto optimization. *Advances in neural information processing systems*, 28.

FDA

- Morris, J.S., 2015. Functional regression. *Annual Review of Statistics and Its Application*, 2(1), pp.321-359.
- Fitzmaurice, G., Davidian, M., Verbeke, G. and Molenberghs, G. eds., 2008. *Longitudinal data analysis*. CRC press.
- Diggle, P.J. and Taylor-Robinson, D., 2024. Longitudinal data analysis. In *Handbook of epidemiology* (pp. 1-34). New York, NY: Springer New York.